Mohsin Iban Hossain
AIUB, Computer Networks Notes

COMPUTER NETWORKS

Table of Contents

What is DHCP?

⇨ **DHCP (Dynamic Host Configuration Protocol)** is a network protocol that automatically gives an IP address to each device (client) in a network.

❖ It helps devices connect to the network without manually configuring IP settings.

❖ **Example:** When you connect your laptop to Wi-Fi, the **DHCP server** gives your device an IP address automatically.

BOOTP (Bootstrap Protocol)

- An older protocol before DHCP.
- Assigns an IP address to devices when they boot up.
- Uses a **static table** — each MAC address is permanently linked to one IP.
- Cannot handle moving devices or temporary IPs.
- **DHCP improved BOOTP** by adding dynamic address allocation.

DHCP Address Allocation

1. **Static Allocation:**
   - IP is permanently assigned to a device (like BOOTP).
2. **Dynamic Allocation:**
   - IP is temporarily assigned from a **pool** for a fixed period (called **lease time**).

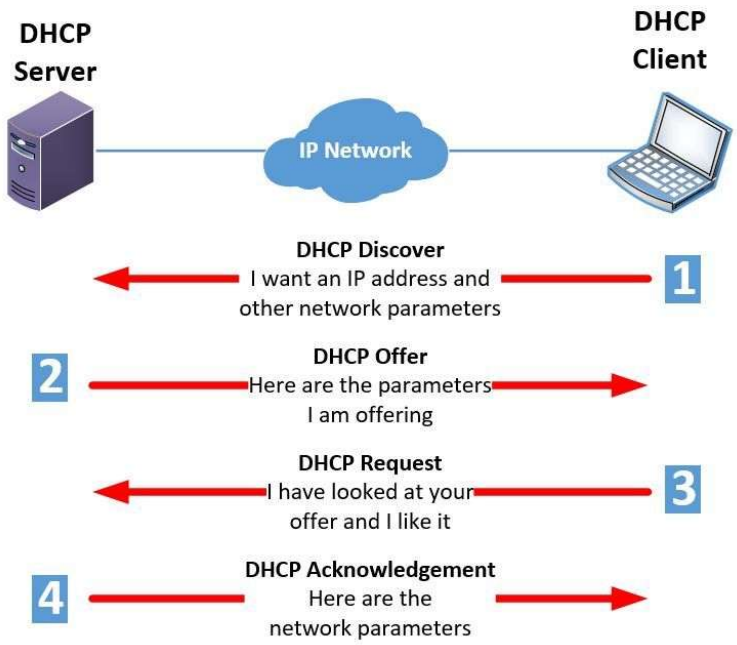
Steps of DHCP Operation

State	Description	Messages Used
INIT	Client starts and broadcasts a request for IP.	DHCPDISCOVER
SELECTING	Servers offer available IPs.	DHCPOFFER
REQUESTING	Client requests one IP offer.	DHCPREQUEST
BOUND	Client uses the IP until the lease expires.	DHCPACK
RENEWING	At 50% lease time, client requests renewal.	DHCPREQUEST + DHCPACK
REBINDING	At 87.5% lease time, client tries again. If no response, it restarts.	DHCPNACK or DHCPACK

❖ **Example:** If lease time (LT) = 1 hour 28 minutes

- Renewing timer = 50% → **44 minutes**
- Rebinding timer = 87.5% → **~1 hour 16 minutes**

Steps of DHCP



1 **DHCP Discover**—The client broadcasts a request to find available DHCP servers, asking for an IP address and other configuration details.

2 **DHCP Offer** – The DHCP server responds with an offer that includes an available IP address, subnet mask, gateway, and DNS settings.

3 **DHCP Request** – The client accepts the offer by sending a request message back to the server.

4 **DHCP Acknowledgement** – The server confirms and finalizes the assignment, and the client is officially connected to the network.

Problem1: A Client pc is looking for an **ip address**. The DHCP server has offered three ip addresses **(207.0.0.6/24, LT- 1 hr 14 min)**, **(207.0.0.8/24, LT- 1 hr 28 min)**, **(207.0.0.11/24, LT- 1 hr 32 min)**. The Client has chosen the **2nd ip address**. If the client asks for extended LT of 22 minutes, What is the new timer of **RENEWING and REBINDING** states? While calculating timer for REBINDING state, consider the lease time was not extended in RENEWING state. [LT means lease time]

Is given,

- Chosen IP address: 207.0.0.8/24
- Original Lease Time (LT): 1 hr 28 min = 88 minutes
- Extended Lease Time: +22 minutes
- New Total Lease Time: `88+22 = 110 min`

We Know,

RENEWING (T1) time:

$$\begin{aligned} \mathbf{T1} &= 50\% \times \text{Total lease time} \\ &= 50\% \times 110 \\ &= 55 \text{ min} \end{aligned}$$

So, the client will start **RENEWING** after **55 minutes**.

REBINDING (T2) time:

The problem states: "while calculating timer for REBINDING state, consider the lease time was **not extended** in RENEWING state."

That means for **T2**, we use the **original lease time** (before extension):

$$\begin{aligned} \mathbf{T2} &= 87.5\% \times \text{lease time} \\ &= 87.5\% \times 88 \\ &= 77 \text{ min} \end{aligned}$$

So, **REBINDING** starts at **77 minutes** (based on the original LT).

What is ARP (Address Resolution Protocol)?

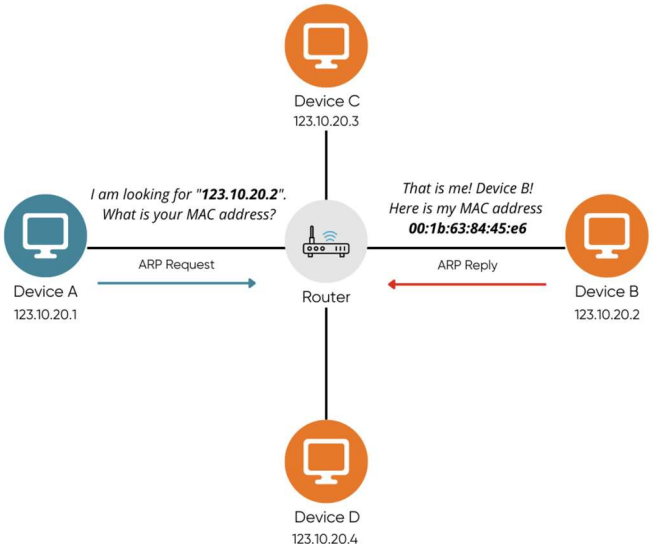
⇨ **ARP** helps a device find the **MAC address** (physical address) of another device using its **IP address** in the same network.

❖ **Example Scenario:**

Device	IP Address	MAC Address
PC-A	123.10.20.1	00:1A:2B:3C:4D:5E
PC-B	123.10.20.2	00:1B:63:84:45:E6
PC-C	123.10.20.3	AC:DE:48:00:11:22
PC-D	123.10.20.4	D4:6D:6D:AA:BB:CC

If PC-A wants to send data to PC-B but only knows its IP → ARP helps find PC-B's MAC address.

How ARP Works

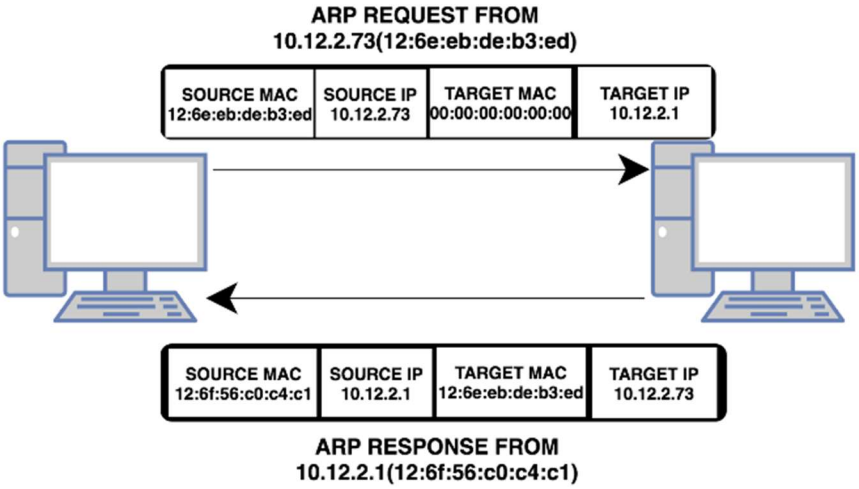


Step 1: ARP Request

- PC-A broadcasts a message asking: "Who has IP **123.10.20.2**? Tell me your MAC address."
- Destination MAC: **00:1A:2B:3C:4D:5E** (broadcast).

Step 2: ARP Reply

- Only PC-B responds: "I am 123.10.20.2. My MAC is **00:1B:63:84:45:E6**."
- PC-A saves this info in its **ARP cache**.



ARP Cache Timeout

- ARP cache stores IP–MAC pairs temporarily.
- Helps avoid sending ARP requests repeatedly.
- Entries **expire after a few minutes** (typically **2–20 mins**) to keep info updated.

Advantages & Disadvantages

Advantages	Disadvantages
Reduces manual IP setup (DHCP).	ARP can be used for attacks (ARP spoofing).
Ensures IP reuse & efficiency.	DHCP failure can stop network access.
Simplifies communication setup.	ARP adds some broadcast traffic.

ERROR CONTROL CODES

Error Control Codes

⇨ During data transmission, **bits may get altered**. **Error Control Codes** detect (and sometimes correct) such errors.

- Two major techniques:
    1. **Cyclic Redundancy Check (CRC)**
    2. **Linear Block Codes**

Cyclic Redundancy Check (CRC)

1. A method for **error detection**.
2. CRC adds extra bits called **Frame Check Sequence (FCS)**.
3. Strength of CRC depends on FCS length, **longer FCS → better error detection**.

❖ Components:

- **Message sequence (M)**
    - The desired data to be sent
    - Can be of any length
    - Also called **Message Polynomial**

- **Pattern sequence (P)**
    - Also called **Generator Polynomial**
    - Known to sender & receiver
    - If FCS has **K bits**, pattern P has $\boldsymbol{N = K + 1}$ **bits**
    - **So,** if pattern P has **N bits, FCS** has $\boldsymbol{K = N - 1}$ bits

CRC Generation at Sender Side

1. Choose number of FCS bits **K**.
2. Append **K zeros** to message M → new sequence **S**.
3. Select a generator pattern **P** (K+1 bits).
4. **Divide S by P** (binary division using XOR).
   Remainder = **FCS (K bits)**.
5. **Remove appended zeros** and append remainder R →
   Transmit sequence T = M + R.

Truth Table of XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

Problem1: Generate FCS if the **message polynomial** and **generator polynomial** are X^3+X^2+1 And X^3+X+1, respectively.

At Sender Side:

Is given,

- $\boldsymbol{M(x)} = X^3 + X^2 + 1 = 1101$
- $P(\boldsymbol{x}) = X^3 + X + 1 = 1011 = 4\ bits$

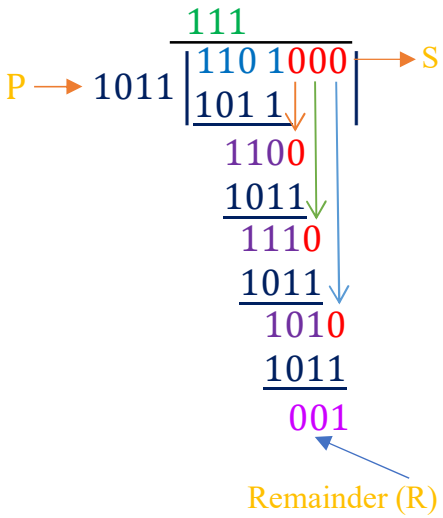
We Know,

$$\mathbf{K} = \mathrm{P} - 1 = 4 - 1 = 3\ bits$$

$$\therefore \mathbf{S} = \text{"}\mathbf{M}\text{"} + \text{"}\mathbf{000}\text{"} = 1101000$$

Now,

Divide S by P



So,

Transmit sequence (T)

$$\mathbf{T} = \text{"}\mathbf{M}\text{"} + \text{"}\mathbf{R}\text{"} = 1101001$$

Error Detection at Receiver Side

1. Receiver divides **received sequence T'** by same pattern **P**.
2. If **remainder = 0** → no error.
3. If **remainder ≠ 0** → error detected → frame rejected.

Problem2: A sender transmitted the codeword obtained from the message polynomial M(x)= X^3+X^2+1 using the generator polynomial P(x)= X^3+X+1. The received bit sequence is **1101001**. Determine whether the received codeword contains an error. Show the remainder and conclude whether the receiver accepts or rejects the frame.

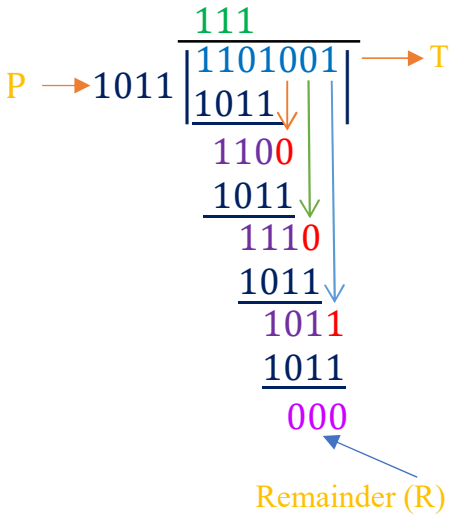
At Receiver Side:

Is given,

- ⇨ $\boldsymbol{M(x)} = X^3 + X^2 + 1 = 1101$
- ⇨ $\boldsymbol{P(x)} = X^3 + X + 1 = 1011 = 4\ bits$
- ⇨ $T \quad = \mathbf{1101001}$

Now,

Divide T by P



∴ Since the remainder is all zeros, the receiver concludes: **NO ERROR → frame accepted.**

Problem3: A sender wants to transmit the message polynomial $M(x) = X^5 + X^2$. using the generator polynomial $P(x) = X^3 + X^2 + 1$.

(a) Generate the **FCS** and the **transmitted codeword**.

(b) At the receiver side, perform the CRC check on the received codeword using the same generator and state whether the frame is **accepted or rejected**.

(a)

At Sender Side:

Is given,

- $\boldsymbol{M(x)} = X^5 + X^2 = 100100$
- $\boldsymbol{P(x)} = X^3 + X + 1 = 1101 = 4\ bits$

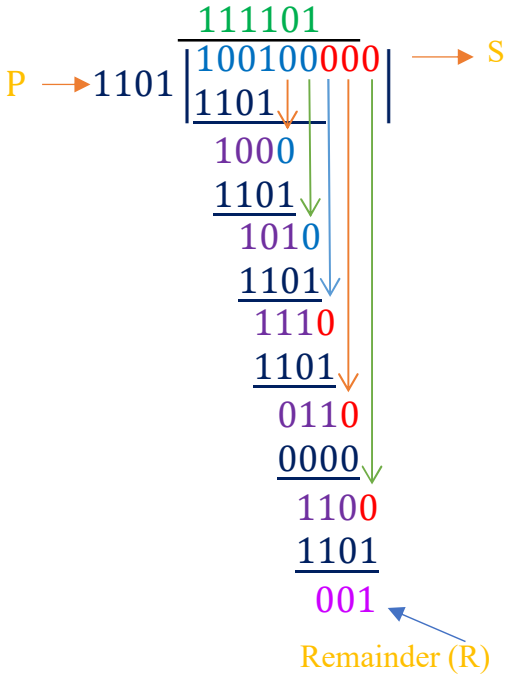
We Know,

$\mathbf{K} = P - 1 = 4 - 1 = 3\ bits$

$\therefore \mathbf{S} = \text{"M"} + \text{"000"} = 100100000$

Now,

Divide S by P



So,

Transmit sequence (T)

$\mathbf{T} = \text{"M"} + \text{"R"} = 100100001$

(b)

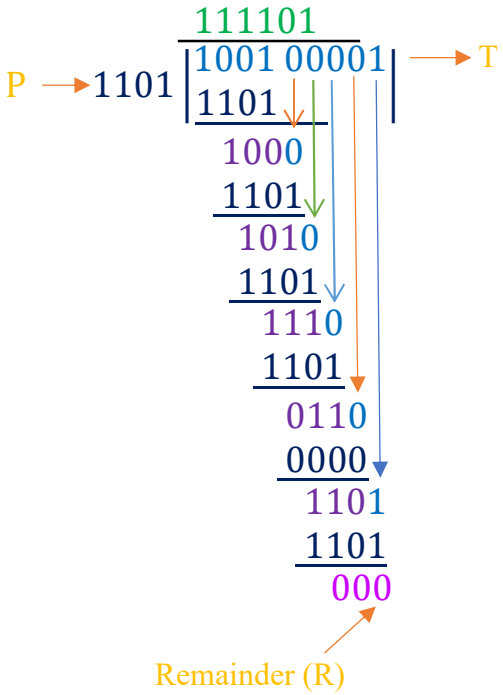
At Receiver Side:

Is given,

- ⇨ $\boldsymbol{M(x)} = X^5 + X^2 = 100100$
- ⇨ $\boldsymbol{P(x)} = X^3 + X + 1 = 1101 = 4\ bits$
- ⇨ $T \quad = 100100001$

Now,

Divide T by P



∴ Since the remainder is all zeros, the receiver concludes: **NO ERROR** → **frame accepted.**

Problem4: Detect whether the received sequence **101110101** is error free if the pattern sequence is **1010**

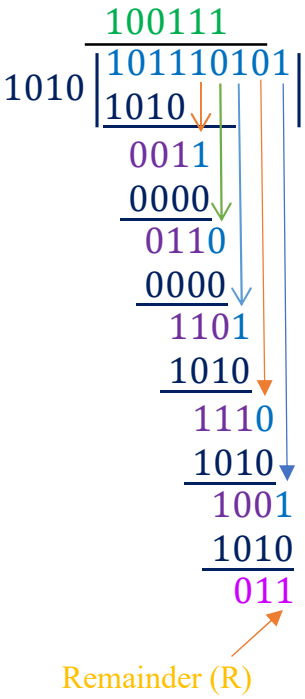
At Receiver Side:

Is given,

- $\boldsymbol{P(x)} = 1010 = 4\ bits$
- $T \quad = 101110101$

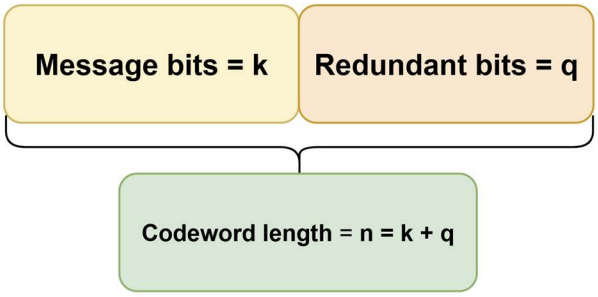
Now,

Divide T by P



∴ Since the remainder is 001 (non-zero),
the received sequence is **NOT error-free** → **Error is detected in the received data**

Linear Block Codes



❖ Basic Terms:

- Message bits = **k**
- Redundant bits = **q**
- Codeword length = **n = k + q**
- Represented using matrices:
  - **Generator Matrix (G)**
  - **Parity Check Matrix (H)**

Generator Matrix (G)

General form:

$$\mathbf{G} = [\mathbf{P_{k\times q}} | \mathbf{I_k}]$$

If the P matrix is:

$$\mathbf{P_{3\times 3}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

& the I matrix is:

$$\mathbf{I_{3\times 3}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

For $k = 3$ & $q = 3$,

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$Then, it\ is\ a\ (\boldsymbol{n}, \boldsymbol{k}) = (\mathbf{6}, \mathbf{3})\ block\ code$$

Codeword Generation

$$\mathbf{C} = \mathbf{M} \times \mathbf{G}$$

❖ **Example message:**

$$\mathrm{M} = [0\ 1\ 1]$$

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

∴ Resulting codeword:

$$\mathbf{C} = \mathbf{M} \times \mathbf{G}$$

$$= [0\ \ 1\ \ 1] \times \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$= \underbrace{\underbrace{[1 \quad 1 \quad 0}_{q} \quad \underbrace{0 \quad 1 \quad 1]}_{k}}_{n}$$

Modulo-2 summation

$0 \times 1 \oplus 1 \times 0 \oplus 1 \times 1 = 1$
$0 \times 1 \oplus 1 \times 1 \oplus 1 \times 0 = 1$
$0 \times 0 \oplus 1 \times 1 \oplus 1 \times 1 = 0$
$0 \times 1 \oplus 1 \times 0 \oplus 1 \times 0 = 0$
$0 \times 0 \oplus 1 \times 1 \oplus 1 \times 0 = 1$
$0 \times 0 \oplus 1 \times 0 \oplus 1 \times 1 = 1$

Parity Check Matrix (H)

General form:

$$\mathbf{H} = [\, \mathbf{I_q} \,|\, \mathbf{P_{k\times q}}^{\mathbf{T}}]$$

If the P matrix is:

$$\mathbf{P_{3\times 3}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

So, the $\mathbf{P}^{\mathbf{T}}$ matrix is:

$$\mathbf{P_{3\times 3}}^{\mathbf{T}} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

& the I matrix is:

$$\mathbf{I_{3\times 3}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

For $k = 3$ & $q = 3$,

$$H = [\, I_q \,|\, P_{k\times q}{}^{T}]$$

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Syndrome Calculation

$$\mathbf{s = r \times H^T}$$

❖ Example1 (error-free case r = C):

⇨ Suppose that there is no error in the received sequence. Hence the received sequence, r, is the same as the transmit sequence, C

$$\text{Received } r = [1\ 1\ 0\ 0\ 1\ 1]$$

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

So, the $\mathbf{H^T}$ matrix is:

$$\mathbf{H^T} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

∴ Resulting Syndrome:

$$\mathbf{s} = \mathbf{r} \times \mathbf{H}^{\mathbf{T}}$$

$$\mathbf{s} = [1\ 1\ 0\ 0\ 1\ 1] \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

Each bit is calculated using **Modulo-2 (XOR) addition.**

$$\mathbf{1 \cdot 1 \oplus 1 \cdot 0 \oplus 0 \cdot 0 \oplus 0 \cdot 1 \oplus 1 \cdot 0 \oplus 1 \cdot 1 = 0}$$

$$\mathbf{1 \cdot 0 \oplus 1 \cdot 1 \oplus 0 \cdot 0 \oplus 0 \cdot 1 \oplus 1 \cdot 1 \oplus 1 \cdot 0 = 0}$$

$$\mathbf{1 \cdot 0 \oplus 1 \cdot 0 \oplus 0 \cdot 1 \oplus 0 \cdot 0 \oplus 1 \cdot 1 \oplus 1 \cdot 1 = 0}$$

$$\mathbf{s} = [0\ 0\ 0]$$

∴ Since the Resulting Syndrome is all zeros, the receiver concludes: **NO ERROR** → **frame accepted.**

❖ Example2 (error case r ≠ C):

⇨ Suppose that there is no error in the received sequence. Hence the received sequence, r, is the same as the transmit sequence, C

$$\text{Received r} = [1\ 0\ 0\ 0\ 1\ 1]$$

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

So, the $\mathbf{H}^{\mathbf{T}}$ matrix is:

$$\mathbf{H}^{\mathbf{T}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

∴ Resulting Syndrome:

$$\mathbf{s} = \mathbf{r} \times \mathbf{H}^{\mathbf{T}}$$

$$\mathbf{s} = [1\ 0\ 0\ 0\ 1\ 1] \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{s} = [0\ 1\ 0]$$

Each bit is calculated using **Modulo-2 (XOR) addition.**

$$\mathbf{1 \cdot 1 \oplus 0 \cdot 0 \oplus 0 \cdot 0 \oplus 0 \cdot 1 \oplus 1 \cdot 0 \oplus 1 \cdot 1 = 0}$$

$$\mathbf{1 \cdot 0 \oplus 0 \cdot 1 \oplus 0 \cdot 0 \oplus 0 \cdot 1 \oplus 1 \cdot 1 \oplus 1 \cdot 0 = 1}$$

$$\mathbf{1 \cdot 0 \oplus 0 \cdot 0 \oplus 0 \cdot 1 \oplus 0 \cdot 0 \oplus 1 \cdot 1 \oplus 1 \cdot 1 = 0}$$

∴ Since the Resulting Syndrome is 010 (non-zero),
the received sequence is **NOT error-free** → **Error is detected in the received data**

How to correct the error?

❖ **Steps:**

1. Calculate **syndrome s**.
2. Find which row of $\mathbf{H}^{\mathbf{T}}$ matches s.
3. That row index = position of error.
4. Flip that bit → corrected codeword.

❖ **For Example 2:**

Step1: Calculate syndrome s	**Step3:** That row index = position of error.
$\mathbf{s} = [0\ 1\ 0]$	2nd bit of the received sequence is erroneous
Step2: Find which row of H^T matches s	**Step4:** Flip that bit → corrected codeword.
$\mathbf{H}^{\mathbf{T}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$	**before flip:** $r = [1\ 0\ 0\ 0\ 1\ 1]$ **after flip:** $r = [1\ 1\ 0\ 0\ 1\ 1]$

Problem5: Consider a **(7, 4) linear block code** whose **generator matrix** is given by:

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

(a) Find the **codeword** corresponding to the **message sequence M= [1 0 1 0]**.

(b) Determine the **parity-check matrix** corresponding to the given generator matrix.

(c) For the received vector r = [1101010], find the **syndrome**. Based on the syndrome, determine whether the received vector is a **valid codeword or not**. If it is erroneous, correct the error.

(a)

Is given,

⇨ $M = 1010$

We Know,

∴ Resulting codeword:

$$\mathbf{C} = \mathbf{M} \times \mathbf{G}$$

$$= [1\ 0\ 1\ \ 0] \times \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \underbrace{\underbrace{0\ \ 0\ \ 1}_{q}\ \ \underbrace{1\ \ 0\ \ 1\ \ 0}_{k}}_{n}$$

(b)

Is given,

- ⇨ $q = 3\ bits$
- ⇨ $\boldsymbol{k} = 4\ bits$

We Know,

General form generator matrix:

$$\mathbf{G} = [\mathbf{P_{k\times q}}|\ \mathbf{I_k}]$$

So, the P matrix is:

$$\mathbf{P_{4\times 3}} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

&, the **I** matrix is:

$$\mathbf{I_{4\times 4}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Now,

General form Parity Check Matrix (H):

$$\mathbf{H} = [\ \mathbf{I_q}|\ \mathbf{P_{k\times q}}^{\mathbf{T}}]$$

If the P matrix is:

$$\mathbf{P_{4\times 3}} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

So, the $\mathbf{P^T}$ matrix is:

$$\mathbf{P_{4\times 3}}^{T} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

For $\boldsymbol{k} = 4$ & $\boldsymbol{q} = 3$,

$$H = [\ I_3|\ P_{4\times 3}{}^{T}]$$

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

(c)

Is given,

Received $r = [1\ 1\ 0\ 1\ 0\ 1\ 0]$

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

So, the $\mathbf{H^T}$ matrix is:

$$\mathbf{H^T} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

We Know,

$\therefore$ Resulting Syndrome:

$$\mathbf{s} = \mathbf{r} \times \mathbf{H^T}$$

$$\mathbf{s} = [1\ 1\ 0\ 1\ 0\ 1\ 0] \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{s} = [1\ 1\ 1]$$

$\therefore$ Since the Resulting Syndrome is 111 (non-zero),
the received sequence is **NOT error-free** → **Received vector is NOT a valid codeword**

❖ To correct the error:

Step1: Calculate syndrome s	**Step3:** That row index = position of error.
$\mathbf{s} = [1\ 1\ 1]$	4^{th} bit of the received sequence is erroneous
Step2: Find which row of H^T matches s	**Step4:** Flip that bit → corrected codeword.
$\mathbf{H^T} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$	**before flip:** $r = [1\ 1\ 0\ 1\ 0\ 1\ 0]$ **after flip:** $r = [1\ 1\ 0\ 0\ 0\ 1\ 0]$

Problem6: For the parity submatrix P given below, check whether the received sequence 111100 is erroneous or not. If it is erroneous, correct the error.

$$\mathbf{P} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

Is given,

⇨ $r = 111100$

We Know,

General form generator matrix:

$$\mathbf{H} = [\mathbf{I_q} | \mathbf{P_{k\times q}}^T]$$

Is given, the P matrix is:

$$\mathbf{P_{3\times 3}} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

So, the $\mathbf{P^T}$ matrix is:

$$\mathbf{P_{3\times 3}}^T = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Since $P_{3\times 3}$ So, $k = 3$ & $q = 3$,

$$H = [I_3 | P_{3\times 3}{}^T]$$

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

So, the $\mathbf{H^T}$ matrix is:

$$\mathbf{H^T} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

We Know,

$\therefore$ Resulting Syndrome:

$$\mathbf{s} = \mathbf{r} \times \mathbf{H^T}$$

$$\mathbf{s} = [1\ 1\ 1\ 1\ 0\ 0] \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{s} = [0\ 0\ 1]$$

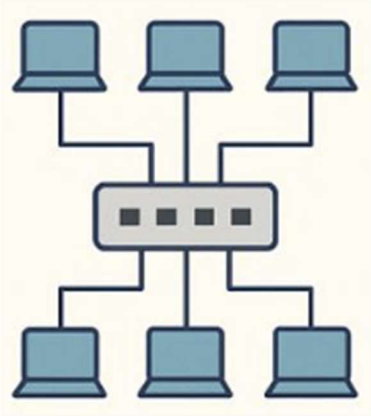
$\therefore$ Since the Resulting Syndrome is 001 (non-zero),
the received sequence is **NOT error-free** → **Received vector is NOT a valid codeword**

❖ To correct the error:

Step1: Calculate syndrome s	**Step3:** That row index = position of error.
$\mathbf{s} = [0\ 0\ 1]$	2nd bit of the received sequence is erroneous
Step2: Find which row of H^T matches s	**Step4:** Flip that bit → corrected codeword.
$\mathbf{H^T} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \\ \mathbf{1} & \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{1} \\ \mathbf{1} & \mathbf{0} & \mathbf{1} \end{bmatrix}$	**before flip:** $r = [1\ 1\ 1\ 1\ 0\ 0]$ **after flip:** $r = [1\ 1\ 0\ 1\ 0\ 0]$

Flat Network

⇨ A **flat network** is a network design that reduces cost and complexity by minimizing routers and hierarchy. It mainly uses hubs and switches.



❖ Characteristics

- Uses a single switch or very few switches
- No hierarchical design
- No separation of broadcast domains
- Devices share the same network

❖ Problems of Flat Network

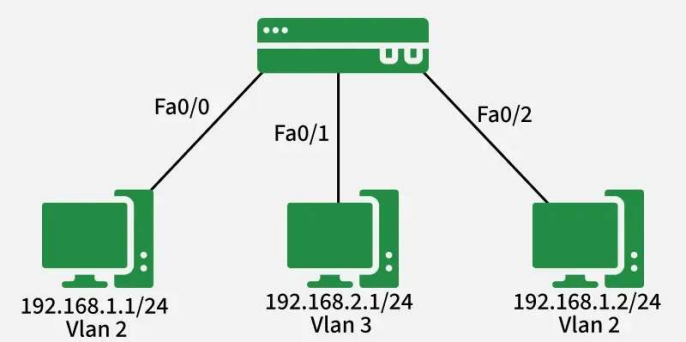
- **Single broadcast domain**
- **Poor network performance** due to heavy broadcast traffic
- **Security issues**
- Network slows down as number of devices increases

Virtual LAN (VLAN)

⇨ A **Virtual LAN (VLAN)** is a logical grouping of devices that behave like they are on the same physical LAN, even if they are connected to different switches.

❖ Purpose of VLAN

- Divide a large, switched network into smaller broadcast domains
- Improve performance and security
- Better network management



❖ Advantages of VLAN

❖ Performance

- Without VLANs, all devices share one broadcast domain
- Broadcast traffic increases as devices increase
- A fault in one switch can disturb the whole network
- VLANs split the broadcast domain
- Each VLAN is a separate broadcast domain
- This **improves network performance**

❖ Security

- VLANs divide a large network into smaller segments
- Unauthorized users cannot easily access other VLANs
- Easier monitoring and control by network administrator

❖ Cost

- Routers are expensive for network separation
- VLANs reduce dependency on routers
- Virtual segmentation lowers overall cost

❖ Location

- Users can be added to a VLAN regardless of physical location

❖ Management

- Easy to assign new users to VLANs using port configuration
- Simplifies network administration

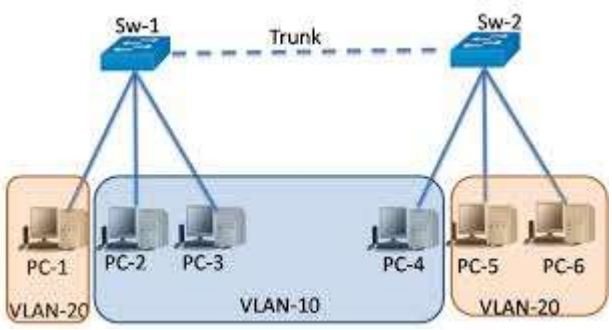
Types of VLAN Connection Links

❖ Access Link

- Carries traffic for **only one VLAN**
- Used between switch and end devices (PCs, printers)

❖ Trunk Link

- Carries traffic for **multiple VLANs**
- Used between switches or between switch and router



Trunk Port

⇨ A **trunk port** is a switch port that carries traffic for multiple VLANs.

❖ Features

- Sends VLAN-tagged frames
- Also called **tagged ports**
- Supported by most managed switches

Standard VLAN tagging protocol

❖ IEEE 802.1Q

- Industry standard VLAN tagging protocol
- Inserts a **4-byte tag** into Ethernet frame
- Identifies which VLAN the frame belongs to
- Recomputes the Frame Check Sequence (FCS)
- Enables trunking between different switches

Trunking

- ⇨ VLANs are local to each switch by default
- ⇨ VLAN information is not automatically shared
- ⇨ Trunk links carry VLAN-tagged traffic between switches

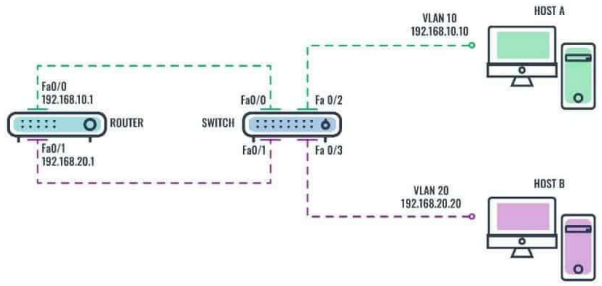
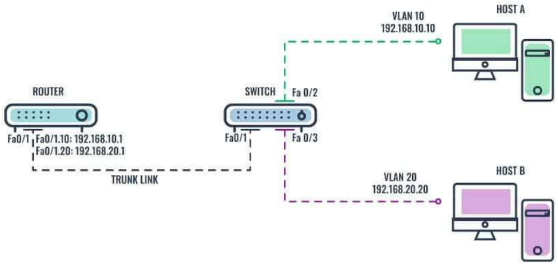
❖ Cisco Trunking Methods

1. **ISL** (Cisco proprietary)
2. **IEEE 802.1Q** (standard)

❖ Trunk Configuration

- Must be enabled on both ends of the link
- Can be configured to allow specific VLANs only

Inter-VLAN Routing

❖ Traditional Inter-VLAN Routing	❖ Router-on-a-Stick
• Router uses **one physical interface per VLAN** • Each interface has its own IP address • Requires more router ports	• Uses **one physical router interface** • Interface is configured as a trunk • Uses multiple sub-interfaces • Cost-effective and scalable • Router receives VLAN-tagged traffic and routes it
	

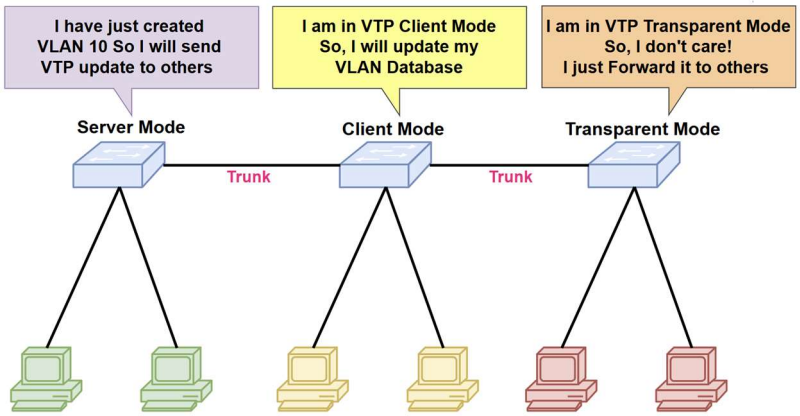
VLAN Trunking Protocol (VTP)

⇨ VTP is a Cisco protocol used to manage VLAN information across multiple switches.

❖ Purpose of VTP

- Centralized VLAN management
- Add, delete, and rename VLANs
- Maintain VLAN consistency across the network

VTP Modes of Operation



❖ Server Mode

- Default mode
- Can create, modify, and delete VLANs
- Advertises VLAN information
- Manages VTP domain settings

❖ Client Mode

- Cannot create or modify VLANs
- Receives VLAN updates from server
- Forwards VTP advertisements

❖ Transparent Mode

- Does not participate in VTP
- Does not advertise VLAN configuration
- VLAN changes affect only local switch
- Forwards VTP advertisements (VTP version 2)

Introduction to Routing Protocols

⇨ A **routing protocol** is used by routers to:

- Discover network destinations
- Select the **best path**
- Maintain routing tables dynamically

EIGRP

⇨ EIGRP (Enhanced Interior Gateway Routing Protocol) is an **advanced distance-vector (hybrid) routing protocol** developed by **Cisco**. It combines the best features of **distance-vector** and **link-state** routing protocols.

RIP vs EIGRP

Feature	RIP	EIGRP
Maximum routers	15 (16 unreachable)	255 (default 100)
Scalability	Low	High
Convergence	Slow	Fast (feasible successor)
Administrative boundary	Not supported	Supported (AS number)
Metric	Hop count	Bandwidth & delay
Algorithm	Bellman-Ford	DUAL
Routing table	Best route only	Best + backup routes
Network size	Small networks	Medium to large networks

EIGRP Metric

⇨ EIGRP uses **composite metric** based on:

- **Bandwidth**
- **Delay**
- Load
- Reliability

⇨ **Default metric uses only:**

- Bandwidth
- Delay

Bandwidth & Delay

Bandwidth	Delay
• Measured in **kbps** • Depends on **interface type** • Config command: `bandwidth <1-10000000>`	• Time for packet to reach destination (theoretical) • Practically set manually • Config command: `delay <10-167772140>`
❖ **Note:** • This is **not real bandwidth** • Used **only for route selection** • Metric uses the **lowest bandwidth** in the route	❖ **Note:** • Unit: microseconds • Metric uses **sum of delays** of all exit interfaces

Default bandwidth and delay

Interface	Bandwidth	Delay (microseconds)
Serial (T1)	1544 Kbps	20,000
Ethernet	10 Mbps	1,000
Fast Ethernet	100 Mbps	100
Gigabit Ethernet	1000 Mbps	10
10 Gigabit Ethernet	10 Gbps	10

EIGRP Metric Calculation

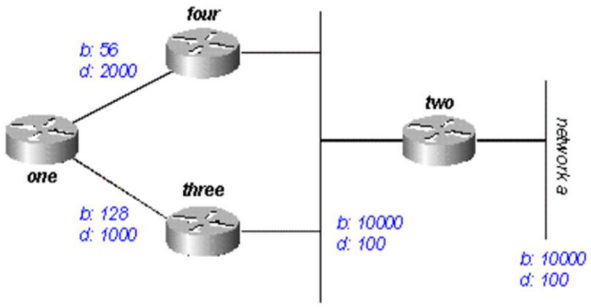
$$\textbf{Metric} = \left[\frac{10^7}{\textbf{Least Bandwidth}} + \textbf{Total Delay}\right] \times 256$$

❖ **Units:**

- **Bandwidth:** kbps
- **Delay:** tens of microseconds
  - Example: if $total\ delay = 30\ \mu s, then$

$$Total\ Delay = \frac{30}{10} = 3$$

Problem1: Calculate metrics for all possible routes from router ONE to network A. Which path will be selected by EIGRP and why?



⇨ **Possible routes,**

1. *Route 1:* $1-4-2-A$
2. *Route 2:* $1-3-2-A$

⇨ **Is given For Route $1-4-2-A$,**

$least\ BW = 56kbps$

$$Total\ Delay = \frac{2000}{10} + \frac{100}{10} + \frac{100}{10} = 220$$

⇨ **We Know,**

$$\textbf{Metric} = \left[\frac{\mathbf{10^7}}{\textbf{Least Bandwidth}} + \textbf{Total Delay}\right] \times \mathbf{256}$$

$$= \left[\frac{10^7}{56} + 220\right] \times 256$$

$$= [178571 + 220] \times 256$$

$$= [178791] \times 256 = 45770496$$

⇨ **Is given For Route 1– 3– 2– *A*,**

$least\ BW = 128kbps$

$$Total\ Delay = \frac{1000}{10} + \frac{100}{10} + \frac{100}{10} = 120$$

⇨ **We Know,**

$$\textbf{Metric} = \left[\frac{10^7}{\textbf{Least Bandwidth}} + \textbf{Total Delay}\right] \times 256$$

$$= \left[\frac{10^7}{128} + 120\right] \times 256$$

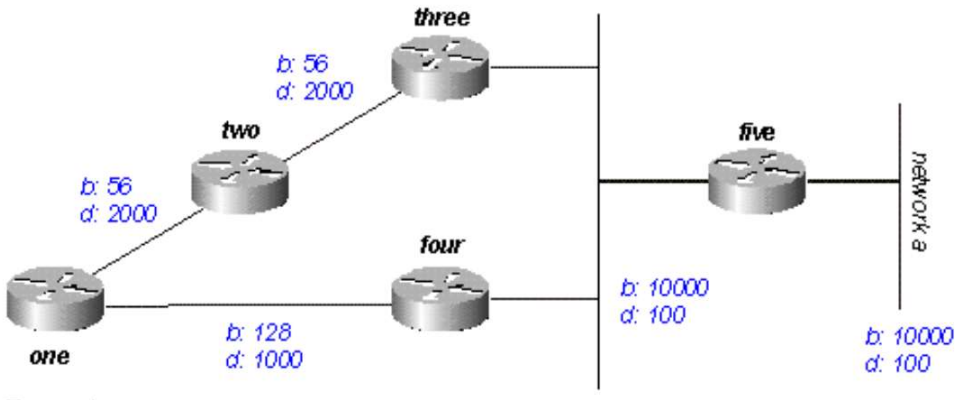
$$= [78125 + 120] \times 256$$

$$= [78245] \times 256 = 20030720$$

➢ **So,**

- **Lower metric is preferred**
- **Route 1–3–2–A** is selected by EIGRP

Problem2: Calculate metrics for all possible routes from router ONE to network A. Which path will be selected by EIGRP and why?



⇨ **Possible routes,**

1. *Route* 1: 1– 2– 3– 5– *A*
2. *Route* 2: 1– 4– 5– *A*

⇨ **Is given For Route 1– 2– 3– 5– *A*,**

$least\ BW = 56\ kbps$

$$Total\ Delay = \frac{2000}{10} + \frac{2000}{10} + \frac{100}{10} + \frac{100}{10} = 420$$

⇨ **We Know,**

$$\textbf{Metric} = \left[\frac{\mathbf{10^7}}{\textbf{Least Bandwidth}} + \textbf{Total Delay}\right] \times \mathbf{256}$$

$$= \left[\frac{10^7}{56} + 420\right] \times 256$$

$$= [178571 + 420] \times 256$$

$$= [178991] \times 256 = 45823232$$

⇨ **Is given For Route 1– 4– 5– *A*,**

$least\ BW = 128kbps$

$$Total\ Delay = \frac{1000}{10} + \frac{100}{10} + \frac{100}{10} = 120$$

⇨ **We Know,**

$$\textbf{Metric} = \left[\frac{\mathbf{10^7}}{\textbf{Least Bandwidth}} + \textbf{Total Delay}\right] \times \mathbf{256}$$

$$= \left[\frac{10^7}{128} + 120\right] \times 256$$

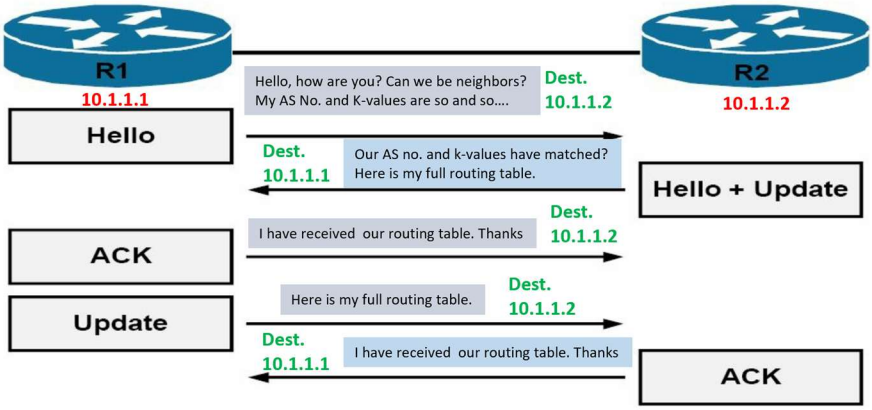
$$= [78125 + 120] \times 256$$

$$= [78245] \times 256 = 20030720$$

➢ **So,**

- **Lower metric is preferred**
- **Route 1–4–5–A** is selected by EIGRP

Neighbor Discovery



EIGRP Neighbor Maintenance

- **Neighbor Discovery**: Full routing table sent initially; afterward, only **changes** are sent.
- **Hello Packets**: Sent periodically to check if neighbor is alive.
  - Default: **every 5 sec** (fast links)
  - Low-bandwidth (T1): **every 60 sec**
- **Hold Time**: Neighbor considered dead if no Hello received within:
  - Default: **15 sec**
  - Low-bandwidth (T1): **180 sec**

❖ Note: Faster links use shorter timers; slower links use longer timers to reduce overhead.

EIGRP Tables & Feasibility Condition

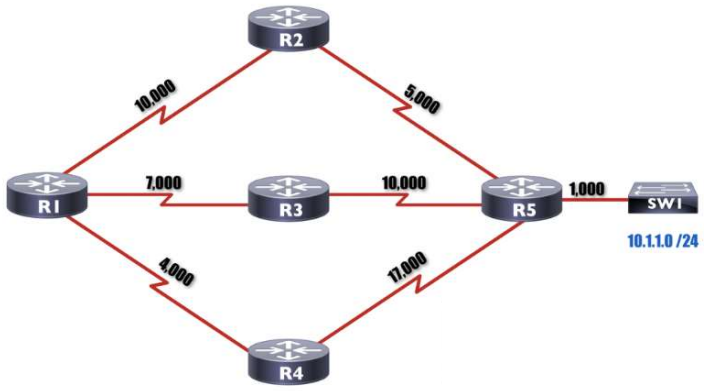
❖ Feasibility Condition (FC):

⇨ A route is considered a **feasible successor** if its **Reported Distance (RD)** is **less than the Feasible Distance (FD)** of the current best path (successor).

⇨ Ensures loop-free backup paths.

Term	Definition
Reported Distance (RD)	Distance advertised by a neighbor to a destination (neighbor → destination).
Feasible Distance (FD)	Sum of **RD** and the distance **from the router to that neighbor**.
Successor	Best path to a destination (lowest FD).
Feasible Successor	Backup path satisfying the **feasibility condition** (RD < FD of successor).

Problem3: The following network topology uses **EIGRP** as the routing protocol. All routers belong to the **same EIGRP Autonomous System**. The destination network **10.1.1.0/24** is connected to **Router R5** through **SW1**. The values shown on each link represent **EIGRP link metrics**.



a) From the perspective of **Router R1**, calculate the **total metric** for all possible paths to reach the destination network **10.1.1.0/24**.
b) Identify the **Successor** route selected by Router R1. Justify your answer.
c) Identify the **Feasible Successor(s)** for Router R1. Clearly state the **Feasibility Condition** used.
d) Identify any routes that are **Not Eligible** to become Feasible Successors and briefly explain why.

(a)

⇨ **Possible routes,**

1. *Route* 1: $1-2-5-A$
2. *Route* 2: $1-3-5-A$
3. *Route* 2: $1-4-5-A$

⇨ **Is given For Route $1-2-5-A$,**

$$\text{Total Metric} = 10{,}000 + 5{,}000 + 1{,}000 = 16{,}000$$

⇨ **Is given For Route $1-3-5-A$,**

$$\text{Total Metric} = 7{,}000 + 10{,}000 + 1{,}000 = 18{,}000$$

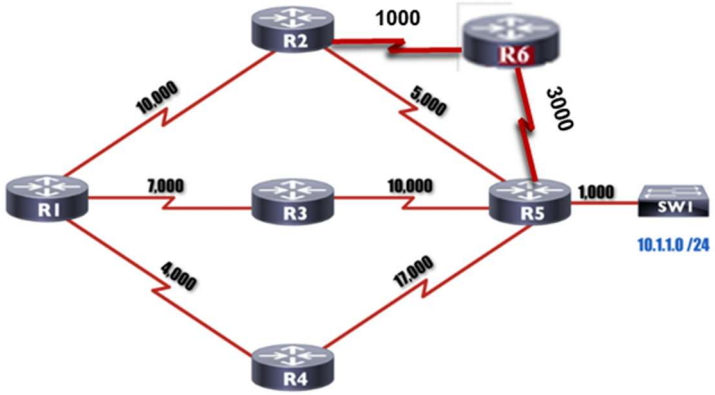
⇨ **Is given For Route $1-4-5-A$,**

$$\text{Total Metric} = 4{,}000 + 17{,}000 + 1{,}000 = 22{,}000$$

(b) & (c) & (d)

Neighbor	RD	FD	(Feasible) Successor?
R2	6,000	16,000	Yes, Successor ($lowest\ FD$)
R3	11,000	18,000	Yes, Feasible Successor ($RD < lowest\ FD$)
R4	18,000	22,000	No ($RD > lowest\ FD$)

Problem4: The following network topology uses **EIGRP** as the routing protocol. All routers belong to the **same EIGRP Autonomous System**. The destination network **10.1.1.0/24** is connected to **Router R5** through **SW1**. The values shown on each link represent **EIGRP link metrics**.



a) From the perspective of **Router R1**, calculate the **total metric** for all possible paths to reach the destination network **10.1.1.0/24**.
b) Identify the **Successor** route selected by Router R1. Justify your answer.
c) Identify the **Feasible Successor(s)** for Router R1. Clearly state the **Feasibility Condition** used.
d) Identify any routes that are **Not Eligible** to become Feasible Successors and briefly explain why.

(a)

⇨ **Possible routes,**

1) *Route 1*: $1-2-5-A$
2) *Route 2*: $1-3-5-A$
3) *Route 3*: $1-4-5-A$
4) *Route 4*: $1-2-6-5-A$

⇨ **Is given For Route 1– 2– 5– *A*,**

$$\text{Total Metric} = 10{,}000 + 5{,}000 + 1{,}000 = 16{,}000$$

⇨ **Is given For Route 1– 3– 5– *A*,**

$$\text{Total Metric} = 7{,}000 + 10{,}000 + 1{,}000 = 18{,}000$$

⇨ **Is given For Route 1– 4– 5– *A*,**

$$\text{Total Metric} = 4{,}000 + 17{,}000 + 1{,}000 = 22{,}000$$

⇨ **Is given For Route 1– 2– 6– 5– *A*,**

$$\text{Total Metric} = 10{,}000 + 1{,}000 + 3{,}000 + 1{,}000 = 15{,}000$$

(b) & (c) & (d)

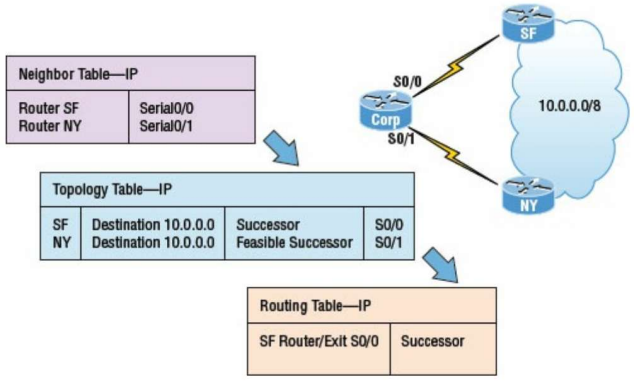
Neighbor	RD	FD	(Feasible) Successor?
R2	5,000	15,000	Yes, Successor ($\boldsymbol{lowest\ FD}$)
R3	11,000	18,000	Yes, Feasible Successor ($\boldsymbol{RD < lowest\ FD}$)
R4	18,000	22,000	No ($\boldsymbol{RD > lowest\ FD}$)

❖ Note: A neighbor can be either Successor OR Feasible Successor — not both.

EIGRP Tables

- **Neighbor Table**: Stores information about **directly connected EIGRP neighbors** (IP, interface).
- **Topology Table**: Stores **all learned routes**, including the **Successor** and **Feasible Successors**.
- **Routing Table**: Stores **only the best route (Successor)** used for **packet forwarding**.

EIGRP Tables: Example



OSPF (Open Shortest Path First)

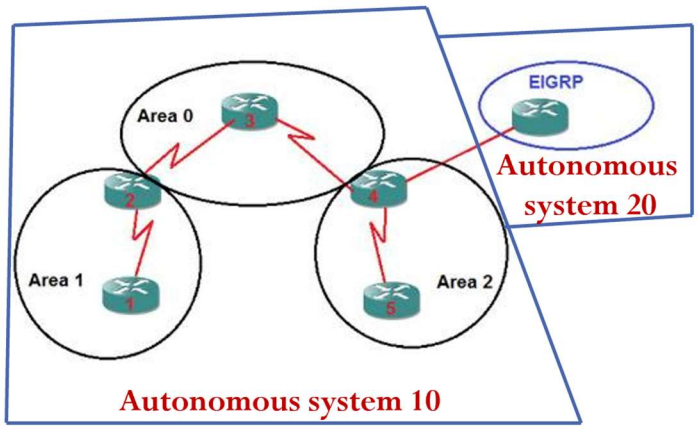
- OSPF is a **link-state interior gateway routing protocol (IGP)** used within a single autonomous system (AS). It finds the **shortest path** to all networks using the **Dijkstra's Shortest Path First (SPF) algorithm**.

EIGRP vs OSPF

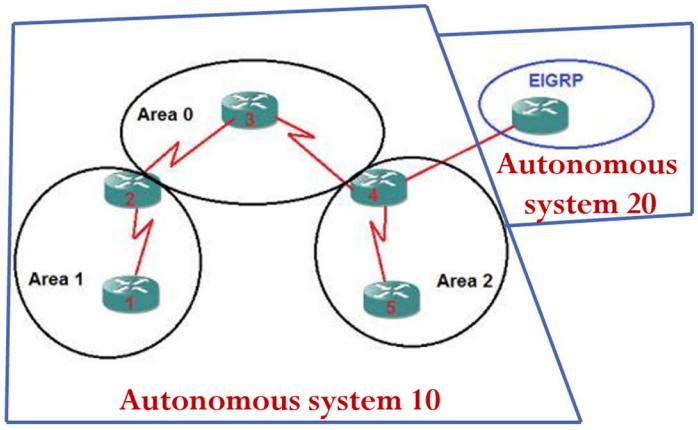
EIGRP	OSPF
Scalable up to 255 routers	Supports unlimited routers
Fast convergence (feasible successor)	Very fast convergence (area-based)
Cisco proprietary	Open standard
Metric: bandwidth & delay	Metric: bandwidth only
Uses DUAL algorithm	Uses Dijkstra SPF algorithm
Maintains best + alternative routes	Maintains best route (all in database)
Administrative distance: 90	Administrative distance: 110
Easy to implement	More complex to implement

OSPF Areas

- An **Autonomous System (AS)** can be divided into **one or more areas**.
- Each area is assigned a unique **Area ID**.
- **Backbone Area (Area 0):**
  - Mandatory for multi-area OSPF.
  - All other areas must connect to Area 0.
- **Routing Information Exchange:**
  - A router exchanges routing info **only with routers in its own area** by default.



OSPF Router Types



1. **Internal Router (IR):**
   - All interfaces belong to **one area**.
   - Example: Router 1, Router 5

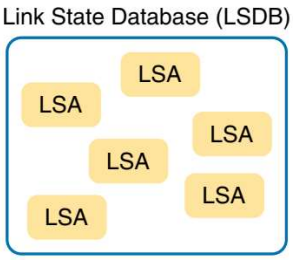
2. **Area Border Router (ABR):**
   - Interfaces in **more than one area**.
   - Example: Router 2, Router 4

3. **Backbone Router:**
   - Has **all or at least one interface in Area 0**.
   - Example: Router 2, Router 3, Router 4

4. **Autonomous System Boundary Router (ASBR):**
   - Connected to a **different autonomous system** (e.g., EIGRP).
   - Example: Router 4

OSPF Data Structures

1. **Link State Advertisement (LSA):**
   - An **LSA** is a message, used by OSPF to **share network information**.
   - Depending on type, holds info about:
     - A router's interfaces
     - All routers attached to a network
     - Summary routing information of an area
     - All routers of an Autonomous System (AS)



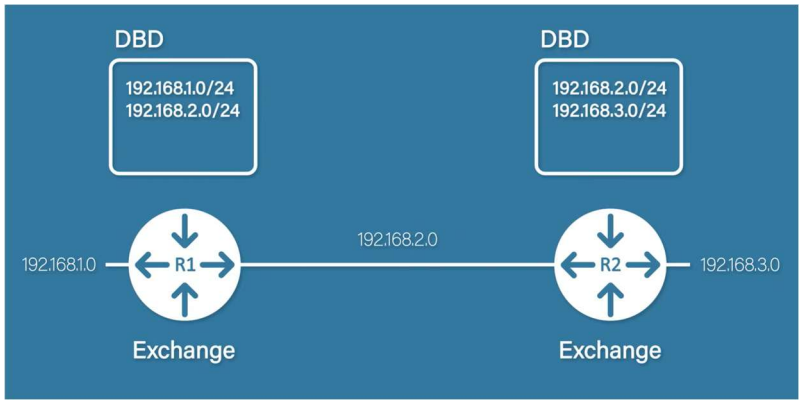
2. **Link State Database (LSDB):**
   - The **LSDB** is a **database that stores all LSAs** known to a router.
   - In a converged network, all routers have the **same LSDB**

OSPF Packet Types

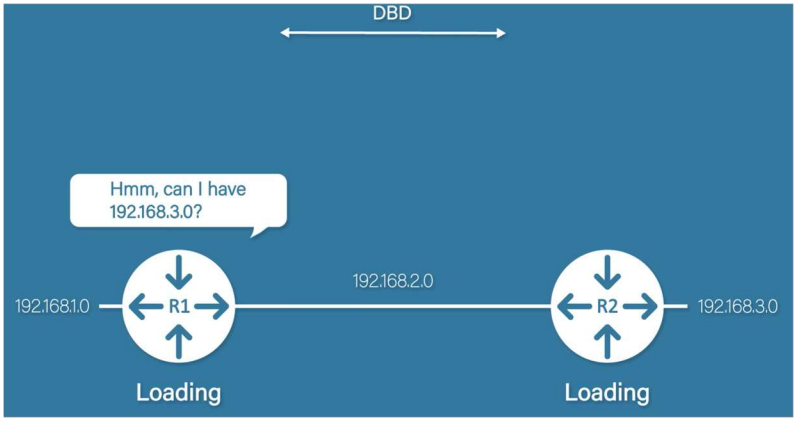
Packet	Purpose
Hello	Build and maintain neighbor relationships
Database Description (DBD)	Lists of LSAs in a LSDB; exchanged during initial database synchronization
Link State Request (LSR)	A Requests for complete information about a specific link from a neighbor
Link State Update (LSU)	Sends one or more LSAs to neighbors
Link State Acknowledgment (LSAck)	Acknowledges reception of an LSA

⇨ An Example:

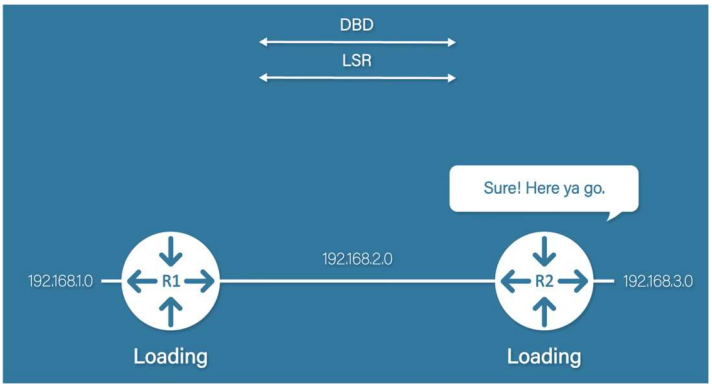
❖ **DBD:** Exchanged during initial database synchronization



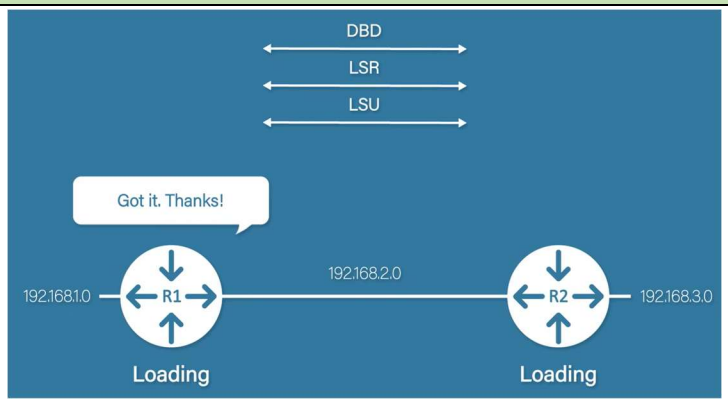
❖ **LSR:** A Requests for complete information about a specific link from a neighbor.



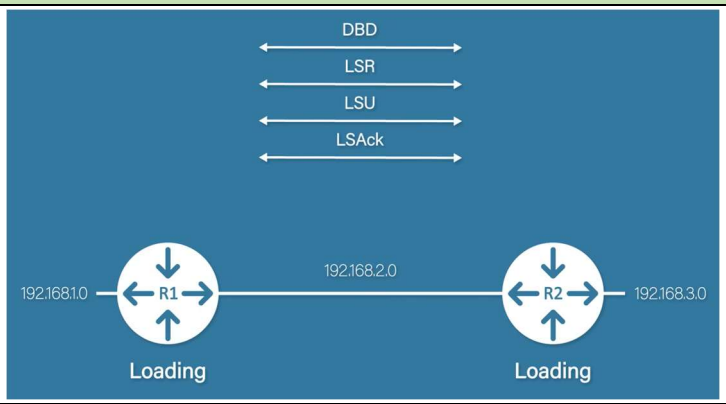
❖ **LSU:** Sends one or more LSAs to neighbors



❖ **LSAck:** Acknowledges reception of an LSA.



❖ **Similar for R2:**



Neighbor Discovery

⇨ For two routers to become **OSPF neighbors**, the following parameters **must match**:

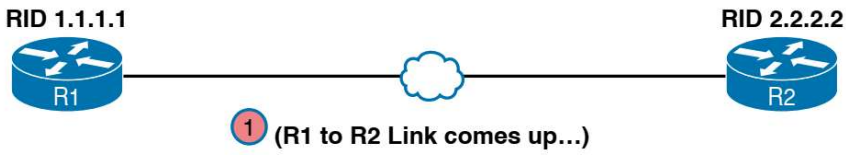
- **Subnet Mask** – Subnet mask of the sending router
- **Subnet Number** – Derived from the subnet mask and interface IP address
- **Area ID** – Area ID of the sending interface
- **Hello Interval** – Time interval between Hello packets
- **Dead Interval** – Time to wait before declaring a neighbor down
- **Authentication Type & Password** – Must match if authentication is enabled
- **Stub Area Flag** – Specifies the stub area type (if used)

❖ **Note:** **Hello packets contain all of the above information** and are used to **discover and maintain neighbor relationships**.

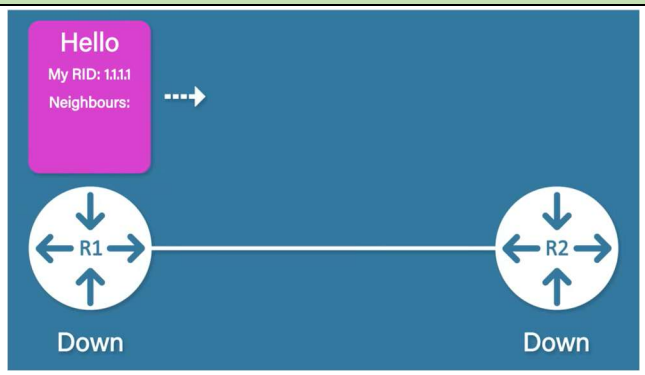
OSPF Neighbor Formation Process

❖ **Link Down State:** Routers have **no knowledge of each other** as OSPF neighbors.

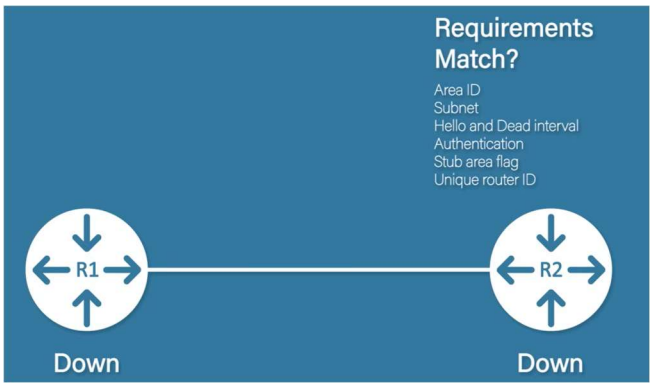
❖ **Link Comes Up (R1–R2):** The physical/link-layer connection between R1 and R2 becomes active.



❖ **Hello from R1:** R1 sends a **Hello packet** to multicast address **224.0.0.5**.

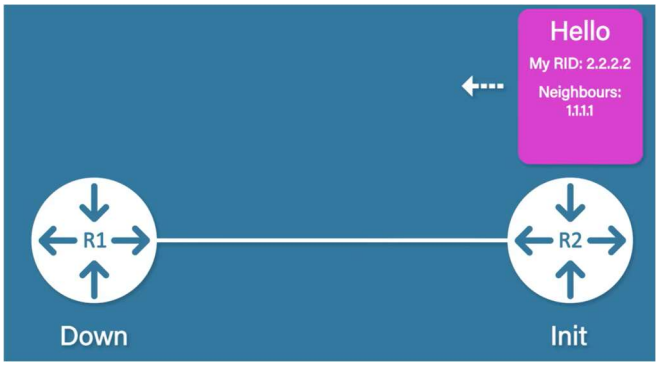


❖ **Requirements Math?:** If all parameters match, R2 accepts R1 as a neighbor.



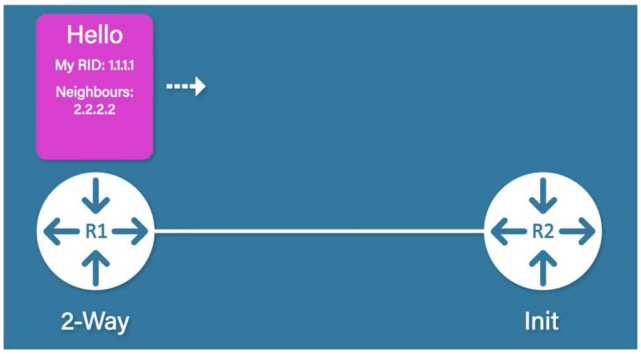
❖ **Init state:** R2 learns about R1 and adds it as a neighbor in the **Init state**.

❖ **Hello from R2:** R2 replies with a Hello packet.



❖ **2-Way state:** R1 now knows R2 exists and moves to the **2-Way state**.

❖ **Another Hello Form R2:** R2 receives the next Hello from R1.



❖ **2-Way State on R2:** R2 also transitions to the **2-Way state**.

Router ID

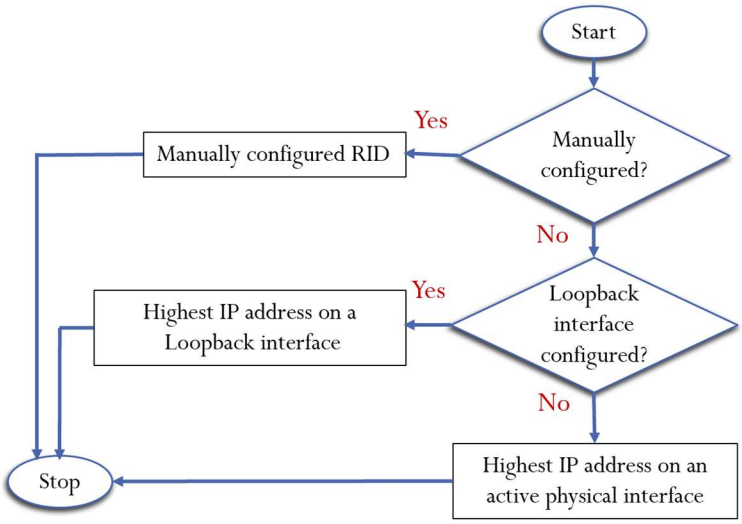
- **Unique 32-bit number** that identifies an OSPF router
- Written in **IP address format** (e.g., `1.1.1.1`)
- **Does not need to be a real IP address**

❖ Router ID Selection Order

⇨ OSPF chooses the Router ID in this order:

1. **Manually configured Router ID**
2. **Highest IP address on a Loopback interface**
3. **Highest IP address on an active physical interface**

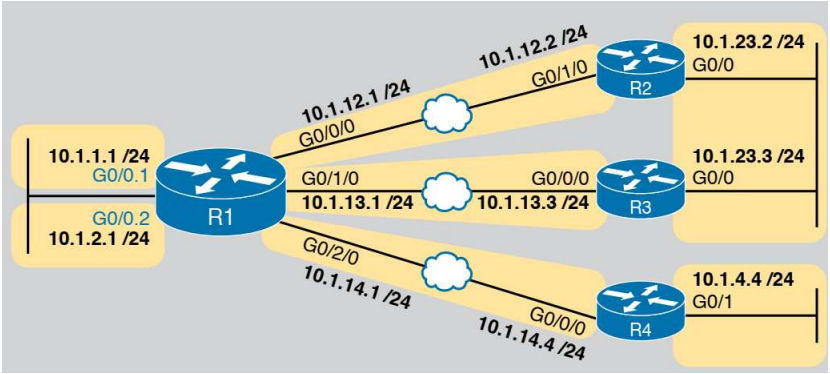
Flow chart of Router ID selection



OSPF Network Types & DR / BDR

1. Point-to-Point Network

- Only **two routers** are directly connected.
- **No DR or BDR election** is needed.
- Routers exchange routing information **directly**.



2. Broadcast Network

- **More than two routers** are connected in the same network.
- Example: Ethernet LAN with many routers.
- If every router exchanged information with all others, it would cause **too much traffic**.

❖ Note: To solve this, OSPF uses **DR and BDR**.

Designated Router (DR)

- A **leader router** in a broadcast network.
- Chosen based on:
    1. **Highest priority**
    2. If tie → **Highest Router ID (RID)**
- **All routers send updates only to the DR**.
- DR then distributes the information to others.

❖ Note: *Reduces network traffic.*

Backup Designated Router (BDR)

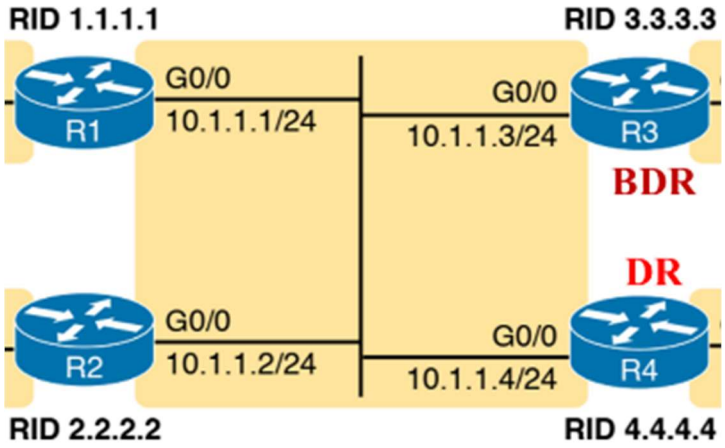
- **Second-in-command router**.
- Chosen by:
    1. Second-highest priority
    2. If tie → second-highest RID
- **Takes over immediately if the DR fails**.

❖ Note: *Ensures reliability.*

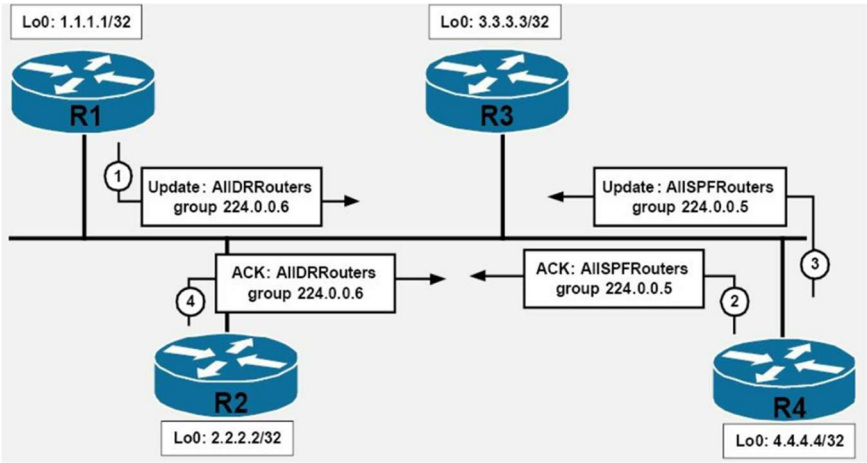
DROTHER

- Routers that are **neither DR nor BDR**.
- They **do not exchange databases directly with each other**.
- They communicate **only with the DR and BDR**.

Broadcast network, DR and BDR election



Broadcast network, DR and BDR election



Wildcard Mask

⇨ A **Wildcard Mask** is used to **specify a range of IP addresses**.

⇨ It is the **inverse of a subnet mask** and is commonly used in:

- **Access Control Lists (ACLs)**
- **EIGRP**
- **OSPF**

Meaning of Each Bit in a Wildcard Mask

Bit Value	Meaning
0	Bit **must match exactly**
1	Bit **can be anything** (0 or 1) also can be ignored

How to Calculate a Wildcard Mask

Wildcard Mask = 255.255.255.255 − Subnet Mask

❖ **Example:** **Subnet Mask:** `255.255.255.0`

Wildcard Mask (decimal):

```
WCM:  255.255.255.255 – 255.255.255.0 = 0.0.0.255
```

Examples

❖ Match ONLY one IP: `192.168.3.0`

- All bits must match

Wildcard Mask (binary):

```
WCM: 00000000.00000000.00000000.00000000
```

Wildcard Mask (decimal):

```
WCM: 0.0.0.0
```

❖ Range: `192.168.3.0 – 192.168.3.255`

- First 24 bits must match
- Last 8 bits can vary

Wildcard Mask (binary):

```
WCM: 00000000.00000000.00000000.11111111
```

Wildcard Mask (decimal):

```
WCM: 0.0.0.255
```

❖ Range: `192.168.3.4 – 192.168.3.13`

Binary: `192.168.3.4 & 192.168.3.13`

```
192.168.3.4:  11000000.10101000.00000011.00000100
192.168.3.13: 11000000.10101000.00000011.00001101
```

Wildcard Mask (binary):

```
WCM: 00000000.00000000.00000000.00001111
```

Wildcard Mask (decimal):

```
WCM: 0.0.0.15
```

Problem5: Calculate the appropriate **wildcard mask** for the following cases.

a. IP address **192.168.3.0**
b. IP address range **192.168.4.0 to 192.168.4.255**
c. IP address range **192.168.4.4 to 192.168.4.13**
d. Subnet mask **255.255.255.240**
e. Network **10.10.5.0/24**
f. Only one host IP **172.16.1.25**
g. IP address range **172.16.8.32 to 172.16.8.63**

(a)

➔ All bits must match

Wildcard Mask (binary):
`WCM: 00000000.00000000.00000000.00000000`
Wildcard Mask (decimal):
`WCM: 0.0.0.0`

(b)

➔ First 24 bits must match. Last 8 bits can vary

Wildcard Mask (binary):
`WCM: 00000000.00000000.00000000.11111111`
Wildcard Mask (decimal):
`WCM: 0.0.0.255`

(d)

Wildcard Mask (decimal):
`WCM: 255.255.255.255 – 255.255.255.240 = 0.0.0.15`

(c)

➔ First 28 bits must match & only Last 4 bits vary

Last octet in binary:

```
4:  00000100
13: 00001101
```

Wildcard Mask (decimal):

```
WCM: 0.0.0.15
```

(e)

➔ Since Network **10.10.5.0/24** so, Subnet Musk: 255.255.255.0

Wildcard Mask (decimal):

```
WCM: 255.255.255.255 – 255.255.255.0 = 0.0.0.255
```

(f)

➔ Since Only one host IP so All bits must match

Wildcard Mask (decimal):

```
WCM: 0.0.0.0
```

(g)

➔ First 27 bits must match & only Last 5 bits vary

Binary: `172.16.8.32 & 172.16.8.63`

```
172.16.8.32: 10101100.00010000.00001000.00100000
172.16.8.63: 10101100.00010000.00001000.00111111
```

Wildcard Mask (binary & decimal):

```
WCM(Binary) : 00000000.00000000.00000000.00011111
WCM(Decimal): 0.0.0.31
```

INTERNET PROTOCOL VERSION 6

Introduction to IPv6

- IPv6 (Internet Protocol Version 6) is the successor to IPv4.
- It was designed to give enough IP addresses for everyone.
- Uses **128-bit addressing**, providing a very large address space.
- Written in **colon hexadecimal notation**.

IPv6 Address Format

- An **IPv6 address** has **8 parts** called **hextets**.
- **Each hextet** has **16 bits** → written as **4 hexadecimal digits**.
- **Total size:** 8 × 16 = **128 bits**.
- **Example:** $\underbrace{2001}_{HN:1}:\underbrace{0211}_{HN:2}:\underbrace{00AB}_{HN:3}:\underbrace{0000}_{HN:4}:\underbrace{0000}_{HN:5}:\underbrace{0000}_{HN:6}:\underbrace{0000}_{HN:7}:\underbrace{0001}_{HN:8}$

❖ **Breakdown of example:**

Hextet No.	Hex Value	Binary Bits	Bits
1	2001	0010 0000 0000 0001	$4+4+4+4=16\ bits$
2	0211	0000 0010 0001 0001	$4+4+4+4=16\ bits$
3	00AB	0000 0000 1010 1011	$4+4+4+4=16\ bits$
4	0000	0000 0000 0000 0000	$4+4+4+4=16\ bits$
5	0000	0000 0000 0000 0000	$4+4+4+4=16\ bits$
6	0000	0000 0000 0000 0000	$4+4+4+4=16\ bits$
7	0000	0000 0000 0000 0000	$4+4+4+4=16\ bits$
8	0001	0000 0000 0000 0001	$4+4+4+4=16\ bits$
Total:			**128bits**

IPv6 Address Rules

Rule No.	Rule	Explanation	Example
1	Omit Leading Zeros	Remove zeros at the **start of a hextet**	0211 → 211, 00AB → AB, 0001 → 1
2	Use Double Colon : :	Replace **consecutive hextets of all zeros** with : : (only once per address)	Original: 2001:0211:00AB:0000:0000:0000:0000:0001 Step 1: 2001:211:AB:0:0:0:0:1 Step 2: 2001:211:AB::1

Problem1: Show the unabbreviated colon hex notation for the following IPv6 addresses.

a. An address with 64 0s followed by 64 1s.
b. An address with 128 0s.
c. An address with 128 1s.
d. An address with 128 alternative 1s and 0s.

(a)

64 0s followed by 64 1s:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
1111111111111111 1111111111111111 1111111111111111 1111111111111111
```

Unabbreviated Addresses:

```
0000:0000:0000:0000:FFFF:FFFF:FFFF:FFFF
```

(b)

128 0s:

```
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
```

Unabbreviated Addresses:

```
0000:0000:0000:0000:0000:0000:0000:0000
```

(c)

128 1s:

```
1111111111111111 1111111111111111 1111111111111111 1111111111111111
1111111111111111 1111111111111111 1111111111111111 1111111111111111
```

Unabbreviated Addresses:

```
FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF
```

(d)

128 alternative 1s and 0s:
1010101010101010 1010101010101010 1010101010101010 1010101010101010 1010101010101010 1010101010101010 1010101010101010 1010101010101010
Unabbreviated Addresses:
AAAA:AAAA:AAAA:AAAA:AAAA:AAAA:AAAA:AAAA

Problem2: Show abbreviations for the following addresses.

a) 0000:0000:FFFF:0000:0000:0000:0000:0000
b) 1234:2346:0000:0000:0000:0000:0000:1111
c) 0000:0001:0000:0000:0000:0000:1200:1000
d) 0000:0000:0000:0000:0000:FFFF:24.123.12.6

(a)

Is Given:	
0000:0000:FFFF:0000:0000:0000:0000:0000	
Step1: Omit Leading Zeros	**Step2: abbreviations Addresses:**
0:0:FFFF:0:0:0:0:0	0:0:FFFF::

(b)

Is Given:	
1234:2346:0000:0000:0000:0000:0000:1111	
Step1: Omit Leading Zeros	**Step2: abbreviations Addresses:**
1234:2346:0:0:0:0:0:1111	1234:2346::1111

(c)

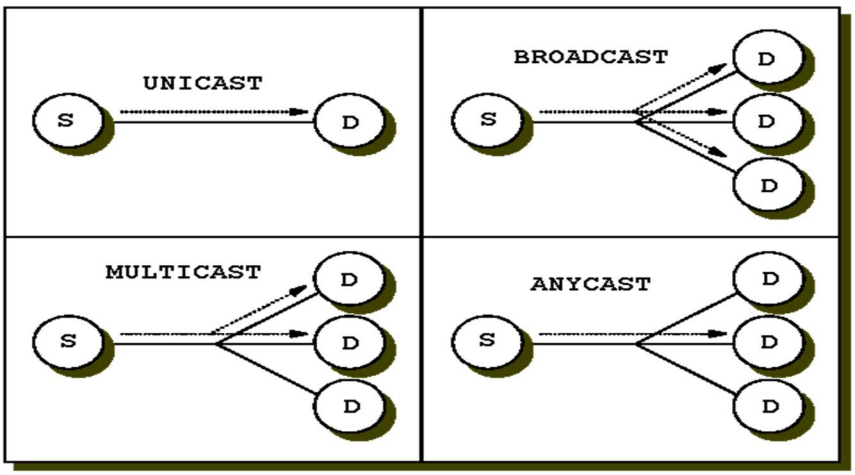
Is Given:	
`0000:0001:0000:0000:0000:0000:1200:1000`	
Step1: Omit Leading Zeros	**Step2: abbreviations Addresses:**
`0:1:0:0:0:0:1200:1000`	`0:1::1200:1000`

(d)

Is Given:	
`0000:0000:0000:0000:0000:FFFF:24.123.12.6`	
Step1: Omit Leading Zeros	**Step2: abbreviations Addresses:**
`0:0:0:0:0:FFFF:24.123.12.6`	`::FFFF:24.123.12.6`

Types of IPv6 Addresses

- **Unicast**: One-to-one communication
- **Anycast**: One-to-nearest destination
- **Multicast**: One-to-many communication
- **Broadcast is removed in IPv6**



What Is Removed in IPv6?

➢ **Broadcast addressing:**

- **Broadcast is not supported in IPv6**
- Broadcast functionality is replaced by **multicast**
- Reduces unnecessary network traffic
- Helps mitigate some **DDoS attacks**

IPv4 to IPv6 Conversion

⇨ IPv6 can represent an IPv4 address using **IPv4-mapped IPv6 addresses**. This is used to **allow IPv6 devices to communicate with IPv4 devices**.

Step 1: Structure of IPv4-Mapped IPv6 Address

Part	Bits	Value
First 96 bits	96	All zeros
Last 32 bits	32	IPv4 address in hexadecimal

Format:

```
::IPv4-in-hex
```

Step 2: Convert IPv4 to Hex

Example IPv4: `192.168.10.62`

Decimal	8 bits Binary	Hextet
192	1100 0000	C0
168	1010 0100	A8
10	0000 1010	0A
62	0011 1110	3E

Step 3: Place IPv4 Hex in IPv6 Address

Take the last 32 bits of IPv6 (8 hex digits) and place the IPv4 hex:

```
C0A8 0A3E
```

Full IPv6 representation with leading 96 bits as zero:

```
0000:0000:0000:0000:0000:0000:C0A8:0A3E
```

Step 4: Remove leading zeros in each hextet.

```
0:0:0:0:0:0:C0A8:A3E
```

Step 5: Replace consecutive all-zero hextets with :: (only once).
& Final IPv6 Address:

```
::C0A8:A3E
```

Problem3: Convert the following IPv4 address to IPv6.

a. 172.16.5.10
b. 10.10.10.20
c. 222.167.34.6
d. 65.180.192.100

(a)

Is Given: IPv4: 172.16.5.10

Decimal	8 bits Binary	Hextet
172	1010 1100	AC
16	0001 0000	10
5	0000 0101	05
10	0000 1010	0A

Place IPv4 Hex in IPv6 Address:

```
AC10 050A
```

Final IPv6 Address:

```
::AC10:50A
```

(b)

Is Given: IPv4: 10.10.10.20

Decimal	Hextet
10	0A
10	0A
10	0A
20	14

Place IPv4 Hex in IPv6 Address:

```
0A0A 0A14
```

Final IPv6 Address:

```
::A0A:A14
```

(c)

Is Given: IPv4: 222.167.34.6

Decimal	Hextet
222	DE
167	A7
34	22
6	06

Place IPv4 Hex in IPv6 Address:

```
DEA7 2206
```

Final IPv6 Address:

```
::DEA7:2206
```

IPv6 Link-Local Address (FE80::/10)

❖ What is a Link-Local Address?

- A **link-local IPv6 address** is used **only within the same local network (link)**.
- It **cannot be routed** outside the local network.
- **All link-local addresses start with:** `FE80::`

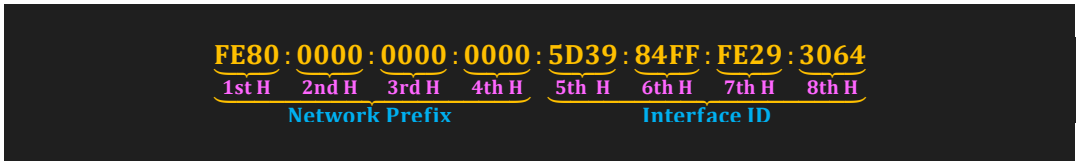
Relationship Between IPv6 Link-Local & MAC Address

⇨ IPv6 uses **EUI-64 format** to generate the **Interface ID (last 64 bits)** from a **48-bit MAC address**.

Example: Given IPv6 Link-Local Address

```
FE80::5D39:84FF:FE29:3064
```

Full expanded form:



❖ Structure Breakdown:

Part	Bits
`FE80::`	Network Prefix (64 bits)
`5D39:84FF:FE29:3064`	Interface ID (Derived from MAC)

Convert IPv6 Link-Local Address → MAC Address

Example: Given IPv6 Link-Local Address

```
FE80::5D39:84FF:FE29:3064
```

Step 1: Drop the First 4 Hextets:

```
Before Drop:- FE80::5D39:84FF:FE29:3064
After Drop:-  5D39:84FF:FE29:3064
              Interface ID
```

Step 2: Convert Interface ID into Bytes & Split into hex pairs

```
5D 39:84 FF:FE 29:30 64
```

Step 3: Remove the Inserted **FF:FE**

In EUI-64: **FF:FE** is **added in the middle** of the MAC address.

```
Before Remove:- 5D 39:84 FF:FE 29:30 64
After Remove:-  5D 39:84 29:30 64
```

Step 4: Flip the 7th Bit of the First Byte (Universal/Local bit):

```
First byte:-  5D
Binary:-      01011101
After Flip:-  01011111
New Hex:-     5F
```

Step 5: Final MAC Address

```
5F39:8429:3064
```

Group into standard MAC format (48bits):

```
5F:39:84:29:30:64
```

Problem4: Convert the following IPv6 Link-Local Addresses to MAC Addresses.

a. FE80::1A2B:3CFF:FE4D:5E6F
b. FE80::F23E:67FF:FE12:9B34
c. FE80::C1D2:ABFF:FE34:5678

(a)

Is Given : IPv6 Link-Local Address: **FE80::1A2B:3CFF:FE4D:5E6F**	
Step 1: Drop the First 4 Hextets	**Step 2:** Remove the Inserted **FF:FE**
1A2B:3CFF:FE4D:5E6F	1A2B:3C4D:5E6F
Step 3: Flip the 7th Bit of the First Byte	**Step 4:** Final MAC Address
First byte:- 1A Binary:- 0001 1010 After Filp:- 0001 1000 New Hex:- 18	182B:3C4D:5E6F

(b)

Is Given : IPv6 Link-Local Address: **FE80::F23E:67FF:FE12:9B34**	
Step 1: Drop the First 4 Hextets	**Step 2:** Remove the Inserted **FF:FE**
F23E:67FF:FE12:9B34	F23E:6712:9B34
Step 3: Flip the 7th Bit of the First Byte	**Step 4:** Final MAC Address
First byte:- F2 Binary:- 1111 0010 After Filp:- 1111 0000 New Hex:- F0	F03E:6712:9B34

(c)

Is Given : IPv6 Link-Local Address: **FE80::C1D2:ABFF:FE34:5678**	
Step 1: Drop the First 4 Hextets	**Step 2:** Remove the Inserted **FF:FE**
C1D2:ABFF:FE34:5678	C1D2:AB34:5678
Step 3: Flip the 7th Bit of the First Byte	**Step 4:** Final MAC Address
First byte:- C1 Binary:- 1100 0001 After Filp:- 1100 0011 New Hex:- C3	C3D2:AB34:5678

Problem5: Convert the following MAC Addresses to IPv6 Link-Local Addresses.

a. 182B:3C4D:5E6F
b. F0E:6712:9B34
c. C3D2:AB34:5678

(a)

Is Given : MAC Addresses: 182B:3C4D:5E6F	
Step 1: Flip the 7th Bit of the First Byte	**Step 2:** Insert FF:FE in the Middle
First byte:- 18 Binary:- 0001 1000 After Filp:- 0001 1010 New Hex:- 1A	1A2B:3CFF:FE4D:5E6F
Step 3: Add FE80:: at the Front	**Step 4:** Final IPv6 Link-Local Addresses
FE80::1A2B:3CFF:FE4D:5E6F	FE80::1A2B:3CFF:FE4D:5E6F

(b)

Is Given : MAC Addresses: F03E:6712:9B34	
Step 1: Flip the 7th Bit of the First Byte	**Step 2:** Insert FF:FE in the Middle
First byte:- F0 Binary:- 1111 0000 After Filp:- 1111 0010 New Hex:- F2	F23E:67FF:FE12:9B34
Step 3: Add FE80:: at the Front	**Step 4:** Final IPv6 Link-Local Addresses
FE80::F23E:67FF:FE12:9B34	FE80::F23E:67FF:FE12:9B34

(c)

Is Given : MAC Addresses: C3D2:AB34:5678	
Step 1: Flip the 7th Bit of the First Byte	**Step 2:** Insert FF:FE in the Middle
First byte:- C3 Binary:- 1100 0011 After Filp:- 1100 0001 New Hex:- C1	C1D2:ABFF:FE34:5678
Step 3: Add FE80:: at the Front	**Step 4:** Final IPv6 Link-Local Addresses
FE80::C1D2:ABFF:FE34:5678	FE80::C1D2:ABFF:FE34:5678

Private Network

⇨ A **private network** is an IP network that is **not directly connected to the Internet**

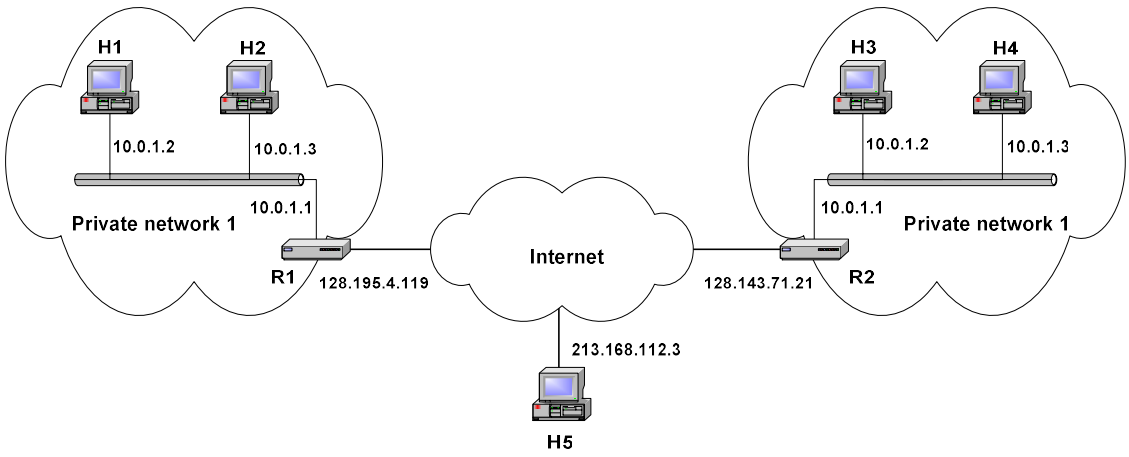
❖ Characteristics:

- IP addresses are **not registered**
- **Not globally unique**
- Cannot be routed on the public Internet
- Used inside organizations (LANs)

❖ Private IP Address Ranges (Non-Routable):

Class	IP Range	CIDR
A	10.0.0.0 – 10.255.255.255	/8
B	172.16.0.0 – 172.31.255.255	/16
C	192.168.0.0 – 192.168.0.255	/24

Private Addresses



Network Address Translation (NAT)

⇨ **NAT** is a technique where a router **replaces private IP addresses with public IP addresses** when packets move between private and public networks and vice versa.

❖ Purpose of NAT:

- Short-term solution to **IPv4 address depletion**
- Saves public IP addresses
- Allows private networks to access the Internet
- Hides internal IP addresses behind a single public IP address.

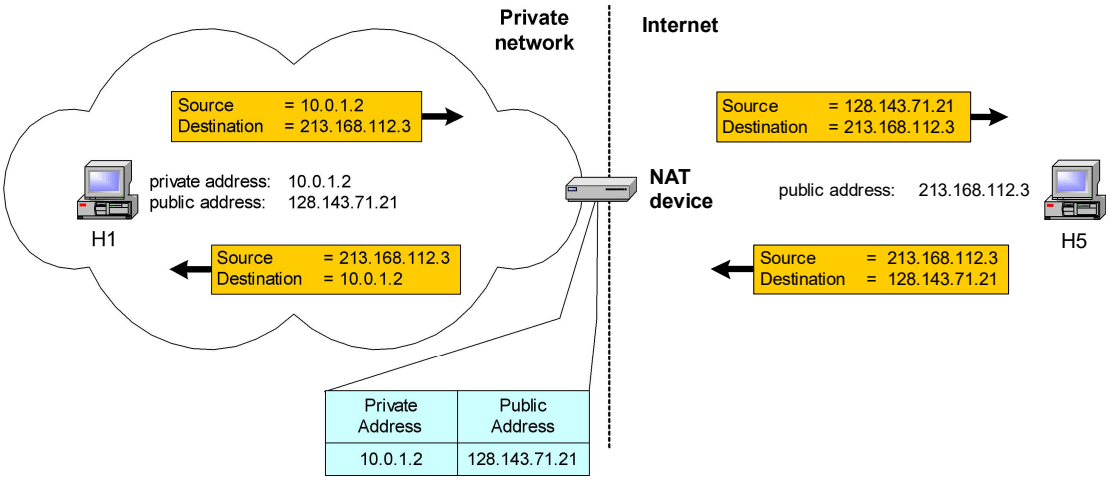
  - Long-term solution: **IPv6**
  - Another solution: **CIDR (Classless Inter-Domain Routing)**

NAT Operation

❖ How NAT Works:

1. Computer uses a **private IP**
2. Packet goes to the **NAT router**
3. Router changes private IP → **public IP**
4. Packet goes to the Internet
5. Reply comes back and NAT sends it to the correct device.

➢ **Note:** NAT keeps a **translation table** to remember who is who.



Pooling of IP Addresses

❖ **Scenario:**

⇨ A corporate network has **many internal hosts** but only a **limited number of public IP addresses** available.

❖ **Solution: NAT with IP Pooling:**

1. **Private Address Space:**
   - The corporate network uses **private IP addresses** (e.g., 10.0.0.0/8,192.168.0.0/16).
   - These addresses are **not routable on the public Internet**.

2. **NAT Device:**
   - A NAT device is placed at the **boundary** between the corporate network and the public Internet.
   - The NAT device manages a **pool of public IP addresses**.

3. **Translation Process:**

⇨ When an internal host accesses the Internet:
   - NAT assigns a **public IP from the pool**.
   - Translates the **source private IP** to this public IP.

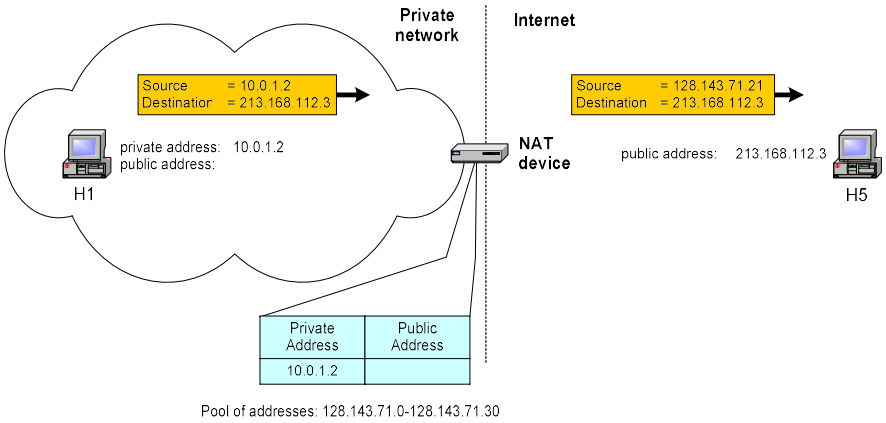
4. **Return Traffic:**
   - Incoming responses are translated back to the **internal private IP** using the NAT **binding table**.

❖ **Benefits:** Saves public IPs, hides internal hosts, allows dynamic use of IPs.

❖ **Example:** NAT binding table.

Private IP	Public IP
192.168.1.10	203.0.113.5
192.168.1.11	203.0.113.6

Pooling of IP Addresses Using NAT



Supporting Migration Between Network Service Providers Using NAT

❖ **Scenario:**

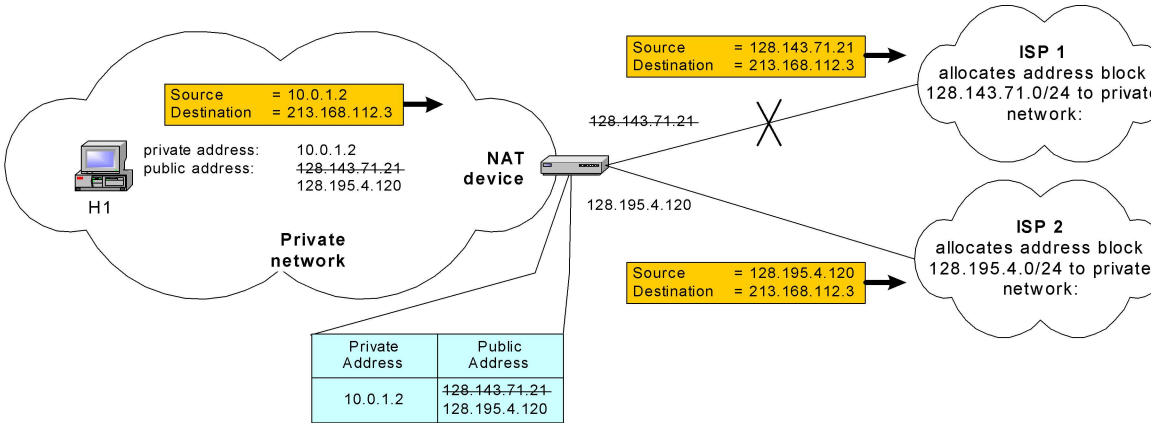
- In CIDR, corporate IPs come from the service provider.
- Changing the provider normally means **changing all IP addresses** in the network.

❖ **NAT Solution:**

- Assign **private IPs** to internal hosts.
- NAT device uses **static translation entries** binding private IPs to public IPs.
- When switching providers, only the **NAT mappings need updating**.
- Hosts continue to use their **private IPs**, so migration is **transparent**.

❖ **Benefit:**

- Simplifies provider migration without reconfiguring internal hosts.



IP Masquerading (NAPT/PAT)

❖ Scenario:

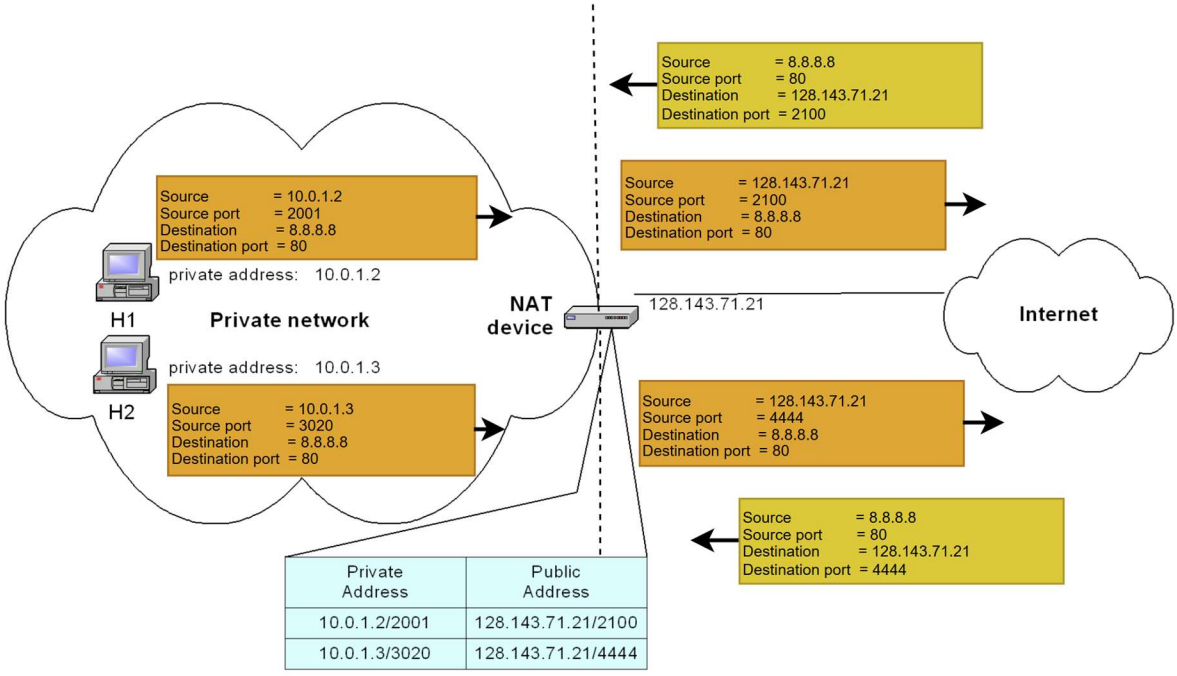
- A **single public IP** serves **multiple private hosts**.

❖ NAT Solution:

- Hosts use **private IPs**.
- NAT device **modifies port numbers** for outgoing traffic, allowing multiple hosts to share the same public IP.

❖ Benefit:

- Saves public IPs while enabling Internet access for many hosts.



Load Balancing of Servers

❖ **Scenario:**

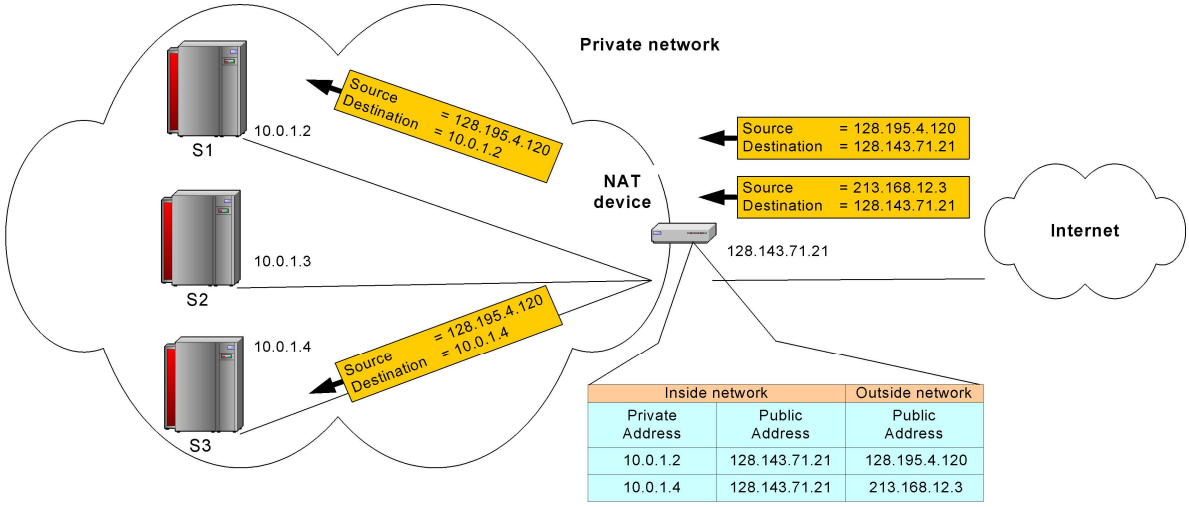
- Multiple **identical servers** provide services using **one public IP address**.

❖ **NAT Solution:**

- Servers are assigned **private IP addresses**.
- NAT device acts as a **proxy** between the public network and servers.
- Incoming packets to the public IP are **translated** by NAT to the **private IP of one server**.
- Load is balanced using strategies like **round-robin assignment**.

❖ **Benefit:**

- Distributes traffic evenly and improves server performance and availability.



Inside network		Outside network
Private Address	Public Address	Public Address
10.0.1.2	128.143.71.21	128.195.4.120
10.0.1.4	128.143.71.21	213.168.12.3

Concerns about NAT

1. Performance Issues:

- When it changes the **IP address**, it must **recheck the packet's safety code** (IP checksum).
- Changing **port numbers** requires **recalculation of the TCP checksum**.
- These extra operations can **slow down packet processing**. especially if many packets arrive at once.

2. Fragmentation Issues:

- Sometimes, a **big message** is **broken into smaller pieces** called fragments.
- NAT must ensure that **all fragments use the same translated IP and port numbers**.
- Otherwise, fragments may be **misrouted or dropped**.

Advantages of NAT

1. **Saves IP Addresses**: Many devices can **share one public IP**, so we don't run out of addresses.
2. **Extra Security**: NAT **hides internal IPs**, so outsiders on the Internet **cannot see your devices directly**.
3. **Flexible Networking**: Makes it easy to **connect private networks to the Internet** without changing much.
4. **Private IP Freedom**: Devices can use **private IPs** and **keep the same addresses** even if the Internet provider changes.

Disadvantage of NAT

- Consumes **CPU and memory resources** due to address translation and maintenance of translation tables.
- May introduce **communication delays** because every packet must be processed by NAT.
- Causes **loss of end-to-end IP traceability**, making troubleshooting and direct communication harder.

TRANSPORT LAYER PROTOCOLS I

Transport Layer

- ⇨ The **Transport Layer** works between the **Network Layer** and the **Application Layer**.
- ⇨ It helps **send messages from one app to another app** (like WhatsApp to WhatsApp).
- ⇨ Ensures **end-to-end (process-to-process) communication**
- ⇨ It makes sure the **whole message arrives safely**. Means **reliable data delivery**
- ⇨ Handles **error control**
- ⇨ Handles **flow control**
- ⇨ It Ensures **correct order of data delivery**
- ⇨ It controls the **speed of data**, so the receiver is not overloaded.

Transport Layer Services

❖ Types of Services

⇨ The transport layer can provide **two types of services**:

1. **Connectionless Service**
2. **Connection-Oriented Service**

What is a Connection?

- A **permanent session** is established before data transfer like making a **phone call** before start talking.
- Devices **prepare to communicate** with each other
- **Traffic amount is negotiated** during session setup means **how much data** can be sent at one time.
- The connection **ends only after** all communication is finished.

Connectionless Service

1. Data is sent **without making a connection first**.
2. Packets sent **independently.**
3. **No packet numbering**
4. Packets may be **lost, delayed, or** May arrive **in the wrong order.**
5. **No acknowledgment**

Connection-Oriented Service

1. A **connection is made first** before sending data.
2. Data is sent **safely and in order**.
3. The receiver sends **acknowledgments**.
4. The connection is **closed after** communication ends.

User Datagram Protocol (UDP)

❖ What is UDP?

- **UDP** is a **connectionless** transport protocol.
- It is **fast but not reliable**.
- Provides **process-to-process communication**.

❖ Main Features of UDP

- No connection is made before sending data.
- Very **little error checking**.
- Does **not remember** the communication session.
- No **flow control** (no speed control).
- Data:
  - May be **lost**
  - May arrive in **wrong order**
- UDP does **not fix mistakes**.
- If reliability is needed, the **application must handle it**.

❖ Why UDP is Fast

- No connection setup.
- No acknowledgments.
- No re-sending of lost data.
- Uses **less control information (overhead)**.

Why Use UDP?

⇨ Used When sending **small messages**.
⇨ Used When **speed is more important than accuracy**.
⇨ Used When the sender **does not care** if some data is lost.

Transmission Control Protocol (TCP)

❖ What is TCP?

- TCP is **connection-oriented** transport protocol.
- It is **slow but reliable**.
- **TCP** sends data from **one program to another program**.
- It uses **port numbers** to send data to the correct app.

❖ Main Features of TCP

- A **connection is made first** before sending data.
- Creates a **virtual connection** between sender and receiver.
- Uses strong **error control** (checks mistakes).
- Uses **flow control** (controls speed).
- Makes sure data is:
  - **Not lost**
  - **In correct order**
- **Delivery of data is guaranteed**.

❖ Speed

- TCP is **slower than UDP**.
- Slower because it:
  - Checks errors
  - Provide acknowledgments
  - Resends lost data
  - Controls speed

TCP Features

1. Numbering System

- TCP gives **numbers every byte** of data.
- It starts with a **random number**.
- This number can be between **0 and $2^{32} - 1$**
- Numbering is **byte-based**, not segment-based.

❖ Example:

- Start number = **1057**
- Total data = **6000 bytes**
- Byte numbers = **1057 to 7056**

2. Sequence Number

- Each TCP segment has a **sequence number**.
- The sequence number is the **number of the first byte** in that segment.
- Helps data arrive in the **correct order**.

3. Flow Control

- TCP controls the **speed of data sending**.
- The **receiver tells the sender** how much data it can handle.
- Prevents the receiver from getting **too much data at once**.

4. Error Control

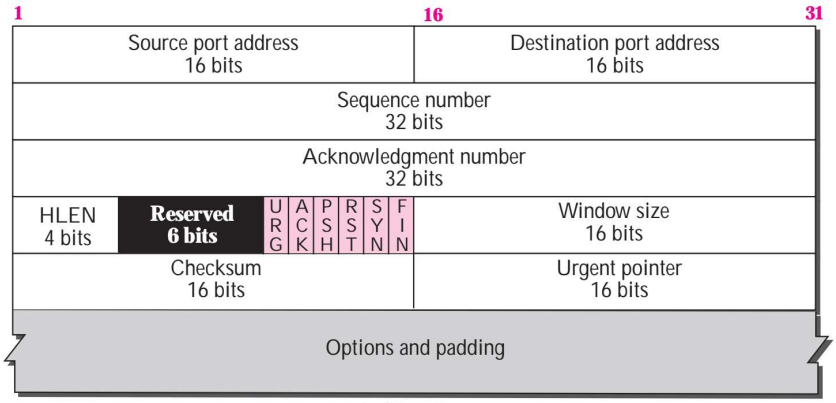
- TCP checks if data is **lost or damaged**.
- If something is wrong, TCP **asks to resend** the data.
- Makes sure data is **correct and complete**.
- Error control is **byte-oriented**

5. Congestion Control

- TCP checks if the **network is busy**.
- If the network is crowded, TCP **slows down**.
- Helps avoid **traffic jams** in the network.

TCP Segment Structure

a. Segment



b. Header

TCP Connection Establishment (3-Way Handshake)

➢ Step 1: Client → Server (SYN)

- The **client sends a SYN segment**.
- Only the **SYN flag** is set.
- Purpose: **Start connection** and **sync sequence numbers**.
- This step **uses 1 sequence number**.
- When data transfer starts, the sequence number increases by **1**.
- **Meaning:** *"Can we talk?"*

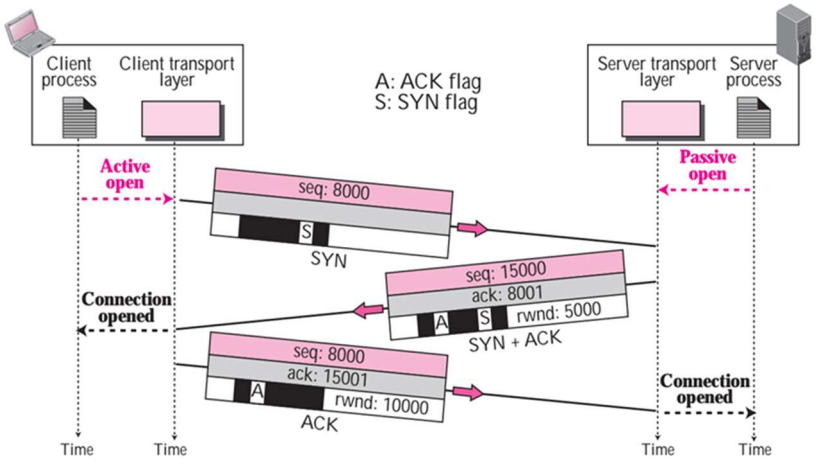
➢ Step 2: Server → Client (SYN + ACK)

- The **server replies with SYN + ACK**.
- **SYN** → Start communication from server side
- **ACK** → Acknowledge client's SYN
- This step also **uses 1 sequence number**.
- **Meaning:** *"Yes, we can talk, and I got your message."*

➢ Step 3: Client → Server (ACK)

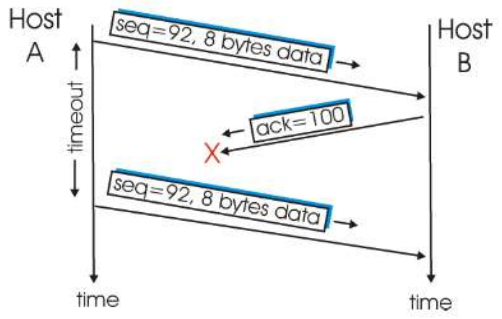
- The **client sends only an ACK segment**.
- Confirms receipt of the server's SYN.
- **No new sequence number is used**.
- Sequence number stays **the same**.
- **Meaning:** *"Okay, let's start sending data."*

Illustration of TCP three-way handshaking



Scenarios of Error Control Mechanisms

❖ Lost Acknowledgment (TCP):



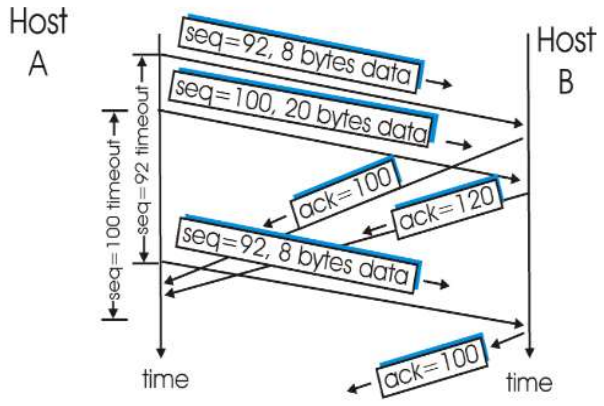
➢ What happens?

1. **Host A sends data** to Host B.
   - Sequence number = **92**
   - Data size = **8 bytes**
   - Bytes sent = **92 to 99**
2. **Host A waits** for an acknowledgment from Host B.
   - Expected ACK number = **100**
3. **Host B receives the data correctly**,
   but the **ACK message gets lost** on the way back.
4. **Host A does not receive ACK**,
   so its **timer expires**.
5. **Host A sends the same data again** (retransmission).
6. **Host B receives the data again**,
   but sees that these bytes are **already received**.
7. **Host B discards the duplicate data**. BUT still sends **ACK = 100 again**

❖ Why does this work safely?

- TCP uses **sequence numbers**.
- Sequence numbers help TCP **identify duplicate data**.
- Duplicate data is **ignored**, not stored again.

❖ Acknowledgment arrives before the timeout:



➢ What happens?

1. **Host A sends two segments one after another**:
   - **Segment 1**
     - Sequence number = **92**
     - Data = **8 bytes** → bytes **92–99**
   - **Segment 2**
     - Sequence number = **100**
     - Data = **20 bytes** → bytes **100–119**

2. **Host B receives both segments correctly**.
3. **Host B sends acknowledgments**:
   - ACK **100** → for first segment
   - ACK **120** → for second segment

4. **Problem occurs**:
   - Both ACKs are **delayed or lost** at first.
   - Host A **does not receive any ACK** before the **timeout of the first segment**.

5. **Timeout happens for Segment 1**:
   - Host A **retransmits Segment 1** (sequence number 92).

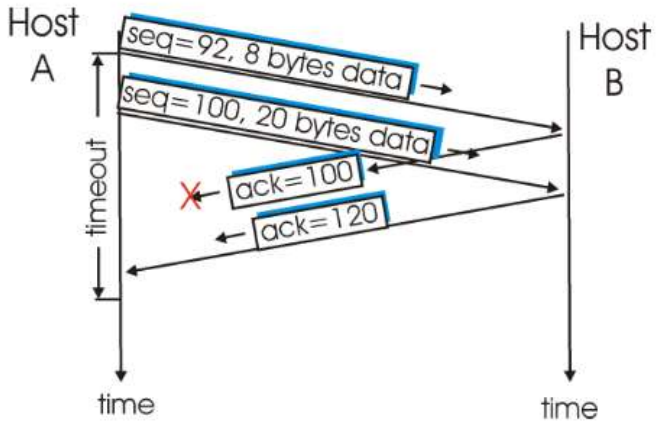
6. **Now an ACK arrives**:
   - ACK **120** arrives **before the timeout of the second segment**.

7. **What does Host A do?**
   - ACK **120** means: "I received **all bytes up to 119**."
   - So Host A **does NOT resend Segment 2**.

❖ Cumulative Acknowledgment:

➢ What is it?

⇨ A **cumulative acknowledgment** means: *One ACK can confirm many segments at once.*



➢ What happens?

1. **Host A sends two segments**:
    - Segment 1:
        - Sequence number = **92**
        - Data = **8 bytes** (92–99)
    - Segment 2:
        - Sequence number = **100**
        - Data = **20 bytes** (100–119)
2. **Host B receives both segments correctly**.
3. **ACK for the first segment is lost**.
4. **Before the timeout**, Host A receives:
    - **ACK = 120**
5. **Meaning of ACK = 120**:
    - Host B has received **all bytes up to 119**.
6. **Result**:
    - Host A **does not resend**:
        - Segment 1
        - Segment 2

Applications of UDP and TCP

❖ Where to Use UDP?	
➢ Use UDP when:	➢ Examples of UDP Applications
• Some **data loss is okay** • **Speed is very important** • Delay must be **very low** • Communication is **simple** (request–reply) • Data goes in **one direction** • Reliability is **not needed** or handled by the app	• **Audio & Video streaming**(VoIP, IPTV) • **DNS** (sometimes uses TCP) • **DHCP** • **SNMP** • **TFTP**
❖ Where to Use TCP?	
➢ Use TCP when:	➢ Examples of TCP Applications
• **Data must not be lost** • Correct **order of data** is important • Delay is **acceptable** • Reliable communication is required	• **HTTP** (web browsing) • **FTP** (file transfer) • **SMTP** (email sending)

TRANSPORT LAYER PROTOCOLS II

Congestion in Networks

❖ What is Congestion?

- **Congestion** happens when **too many packets** try to use the network at the same time.
- The network **cannot handle** all packets.
- It is like a **traffic jam**.
- If $Packets > Network\ capacity = Congestion$

❖ Congestion Control:

- **Congestion control** means **reducing traffic** so the network works smoothly.
- It keeps the network $\boldsymbol{load} < Network\ capacity$
- Used to **avoid packet loss and delay**.

❖ Why Congestion Happens:

- Routers and switches use **queues (buffers)** to store packets.
- Each router interface has:
  - **Input queue** → packets waiting to be processed
  - **Output queue** → packets waiting to be sent

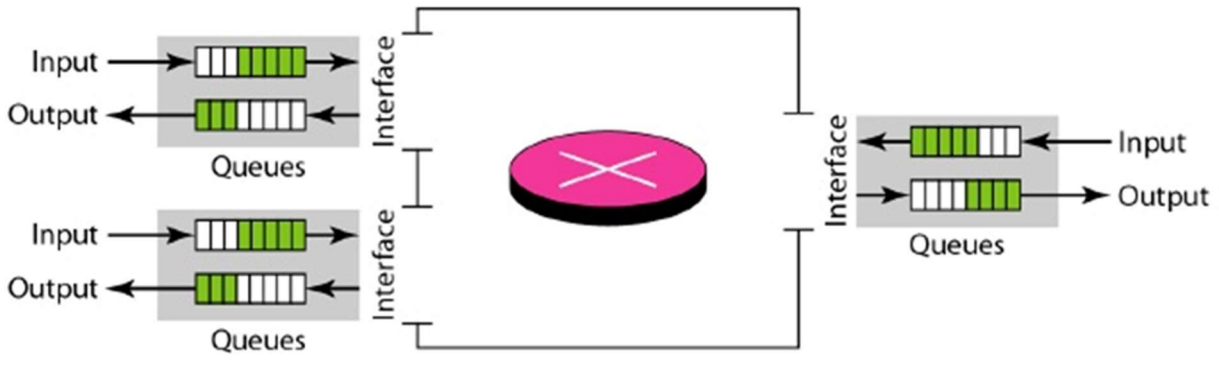
❖ How a Packet Moves in a Router

1. Packet enters and waits in the **input queue**.
2. Router **checks the destination** using the routing table.
3. Packet is placed in the **output queue** and waits to be sent.

Congestion Situations

❖ Two Congestion Situations

1. Input Queue Congestion	2. Output Queue Congestion
• Packet arrival rate > packet processing rate • Input queue gets **longer and longer**	• Packet departure rate < packet processing rate • Output queue gets **longer and longer**
Means: Router cannot process packets fast enough	Means: Router cannot send packets fast enough



Congestion Control

➢ Congestion Window (cwnd):

⇨ **cwnd** = how much data the sender can send **without waiting for ACK**.

⇨ Measured in **bytes**.

❖ Rule:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

⇨ Sending Rate:

- The **sending rate** depends on the congestion window and RTT.
- By adjusting **cwnd**, the sender controls how fast data is sent into the network.

❖ Formula:

$$\text{Sending Rate} = \frac{\text{cwnd}}{\text{RTT}}$$

Where,

RTT = Round Trip Time

Loss Event (Segment Lost)

⇨ The sender detects **lost segments** in two main ways:

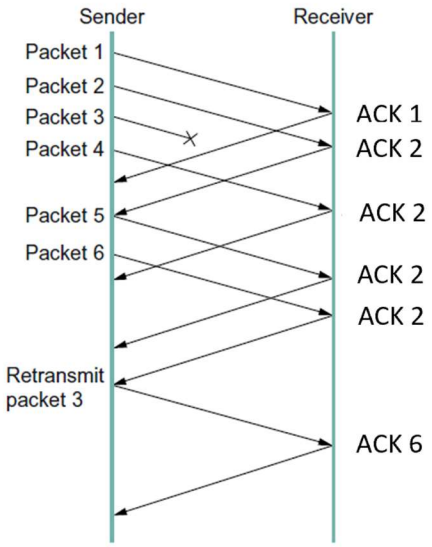
a) Timeout

- The sender waits for an **ACK** from the receiver.
- If **no ACK arrives within a predefined time**, the sender assumes the segment is lost and **retransmits** it.

b) 3 Duplicate ACKs

- Suppose the sender sends segments 1, 2, 3, 4, 5, 6, …
- Segment 3 gets lost, but 4, 5 & 6 arrive at the receiver.
- The **receiver** sends ACK 2 multiple times (duplicate ACKs) because it's still waiting for segment 3.
- The sender sees **3 duplicate ACKs** and realizes:
  "Segment 3 is lost, but the network is not congested. I'll just retransmit segment 3."
- Receiver is receiving segment wrong order.

TCP Loss Detection via Duplicate Acknowledgements



TCP Reno Congestion Control

⇨ TCP Reno controls how the **sender** increases the **congestion window (cwnd)** so it doesn't overload the network.

❖ It has **two main phases**:

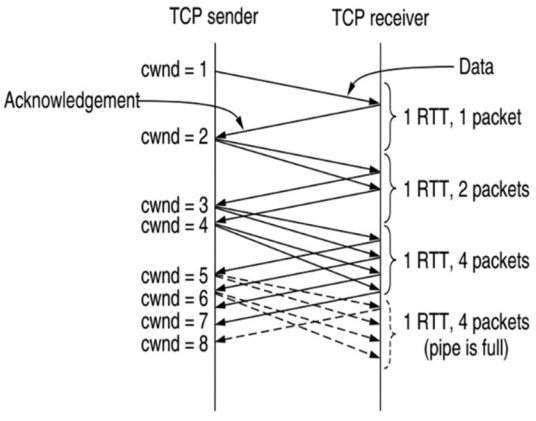
1. Slow Start
2. Congestion Avoidance

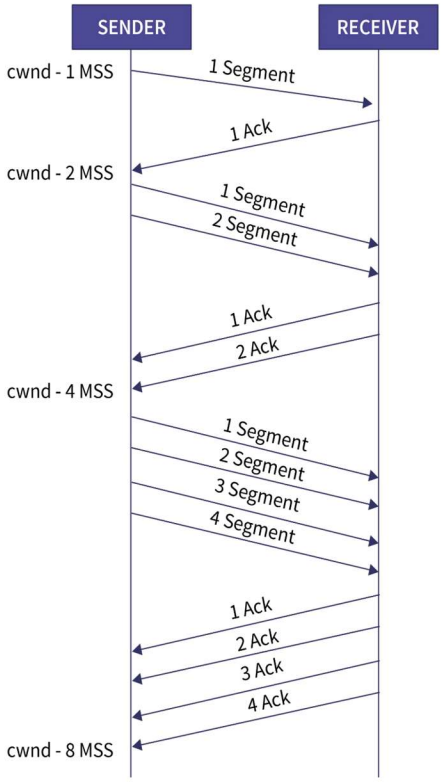
Slow Start

⇨ Follows a greedy approach.
⇨ Starts **small and grows fast** (exponential growth).
⇨ **Sender** begins by sending **1 MSS (Maximum Segment Size)**.
⇨ For **every ACK** received, cwnd **doubles** in the next round.

❖ Step by Step Example:

1. Send 1 MSS → Receive 1 ACK → next round: send 2 MSS
2. Receive 2 ACKs → next round: send 4 MSS
3. Receive 4 ACKs → next round: send 8 MSS …





❖ **Rule:**

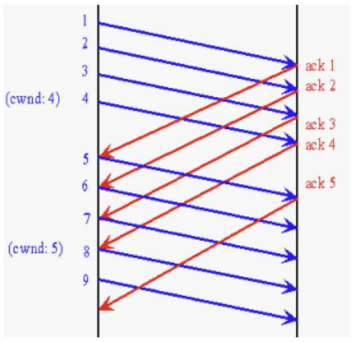
If you send **N** **MSS** last round and get **M** **ACKs** before this round
→ send **N** + **M** **MSS** this round

❖ **Think of it like:**

"Senders send more and more data **very quickly**
until the network shows signs of congestion."

Congestion Avoidance

⇨ When cwnd reaches a **threshold** (ssthresh), slow start **stops exponential growth**.
⇨ Now cwnd **increases slowly (linearly)** → 1 MSS per round-trip time (RTT).



❖ **Rule:**

If cwnd >= ssthresh (***threshold***)
→ *increase cwnd by* **1** ***MSS*** *per RTT*

This prevents the network from being overloaded.

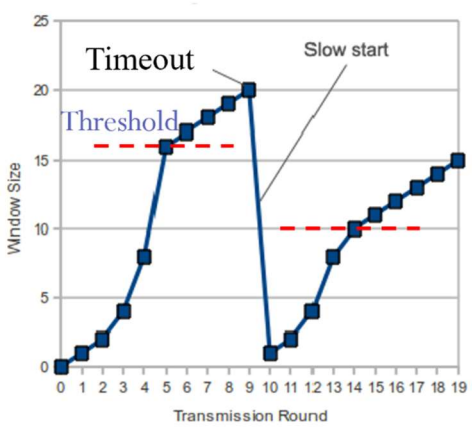
Handling Network Problems

⇨ There are **two ways TCP Reno knows something went wrong**:

1. Timeout
2. 3 Duplicate ACKs

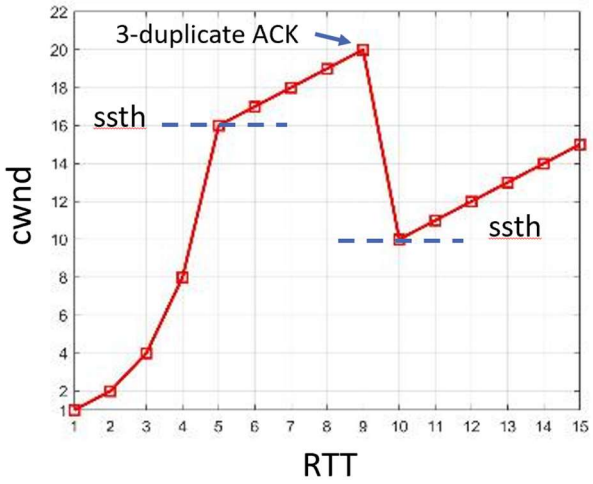
Timeout

- ⇨ Sender does **not get ACK in time** → segment lost.
- ⇨ Action:
  - $threshold = cwnd / 2$
  - $cwnd = 1\ MSS$
  - Go back to **Slow Start**



3 Duplicate ACKs

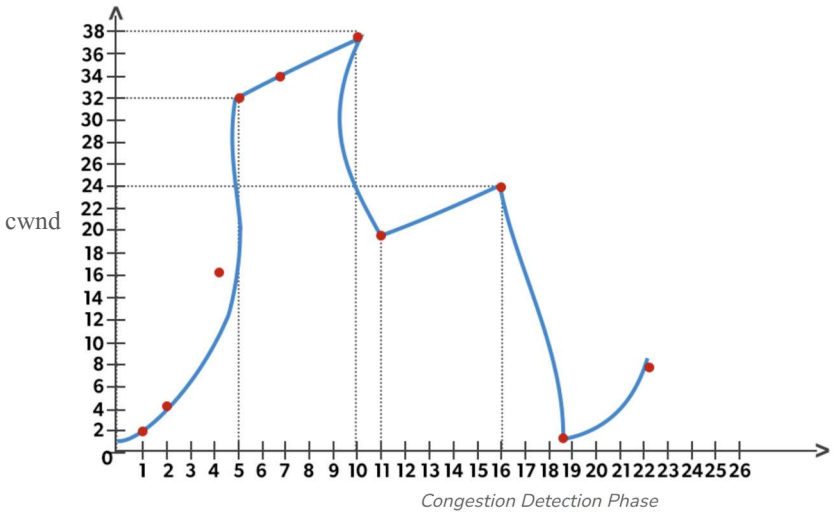
- ⇨ Receiver keeps sending **same ACK 3 times** → one segment lost but network is okay.
- ⇨ Action:
  - $threshold = cwnd / 2$
  - $cwnd = threshold$
  - Go to **Congestion Avoidance**



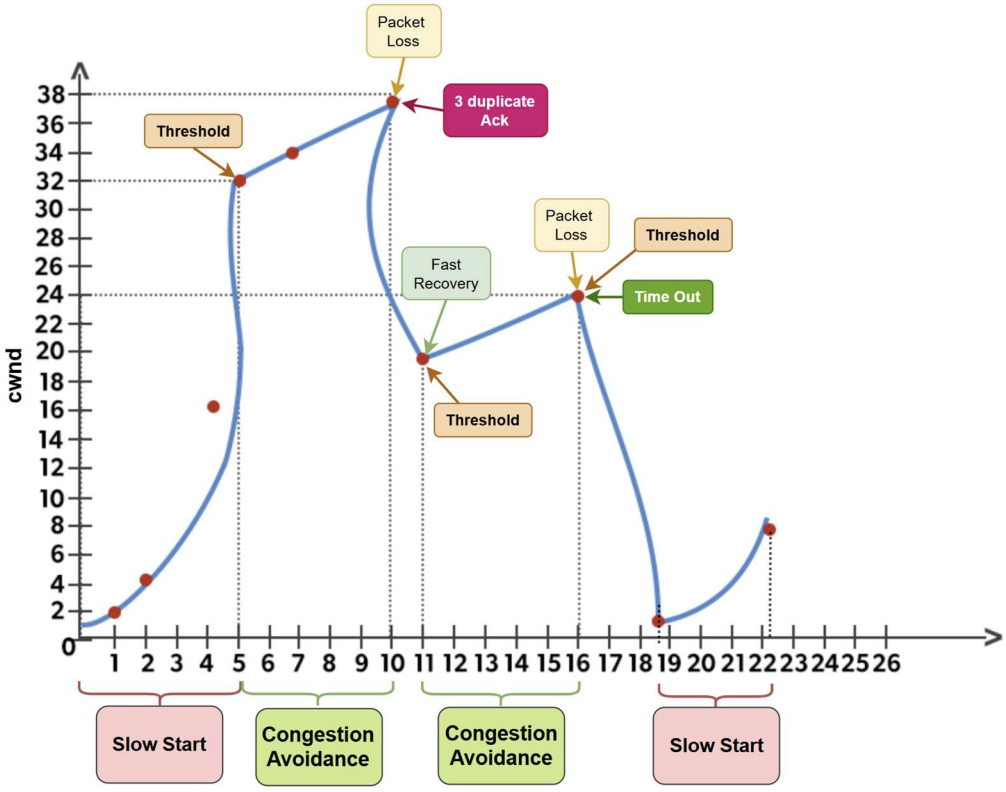
Problem1: Consider the figure below. Assuming TCP Reno is the protocol experiencing the behavior shown above, answer the following questions. In all cases, you should provide a short discussion justifying your answer.

a) Find the **Events** in the TCP RENO.

b) What are The Values Of **Ssthresh** And **Cwnd** at each event?



(a)



(b)

⇨ Values of **`ssthresh`** and **`cwnd`** at Each Event:

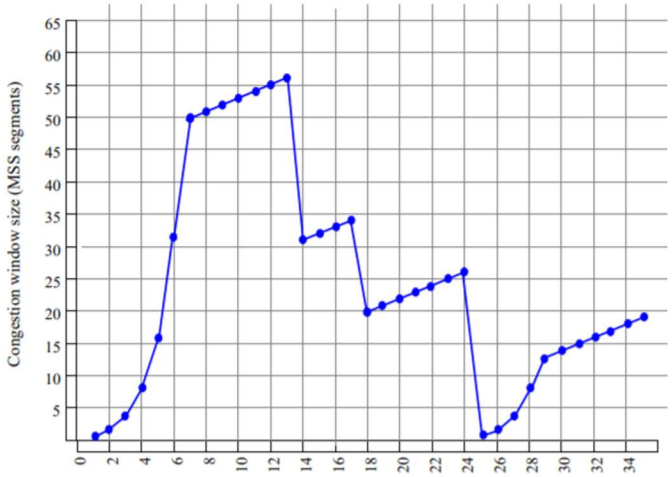
Event	Event Description	ssthresh	cwnd
1	Slow Start begins	32	1 → 32
2	CA starts	32	32 → 38
3	3 Dup ACK	$\frac{cwnd}{2} = \frac{38}{2} = 19$	$cwnd = ssth = 19$
4	CA resumes	19	19 → 24
5	Timeout	$\frac{cwnd}{2} = \frac{24}{2} = 12$	$cwnd = 1$
6	Slow Start	12	1 → 8

⇨ A TCP connection starts in **slow start**. By the **5th round**, the congestion window reaches the **ssthresh (32)** and switches to **congestion avoidance** until the **10th round**. At round 10, **3 duplicate ACKs** trigger **congestion avoidance** again. At **round 16, a timeout occurs**, reducing the window & again starts **slow start**.

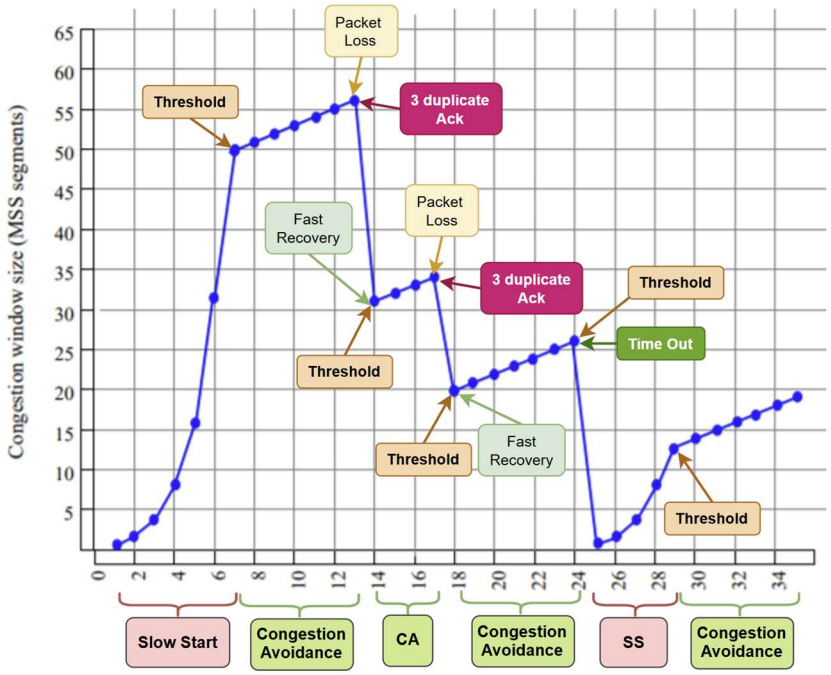
Problem2: Consider the figure below. Assuming TCP Reno is the protocol experiencing the behavior shown above, answer the following questions. In all cases, you should provide a short discussion justifying your answer.

a) Find the **Events** in the TCP RENO.

b) What are The Values Of **Ssthresh** And **Cwnd** at each event?



(a)



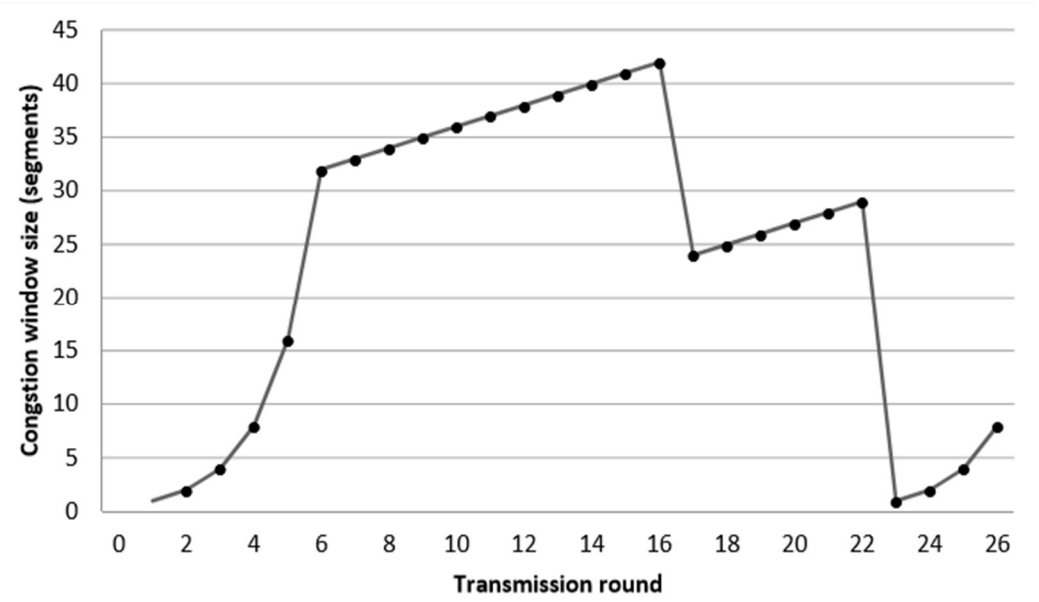
(b)

⇨ Values of **`ssthresh`** and **`cwnd`** at Each Event:

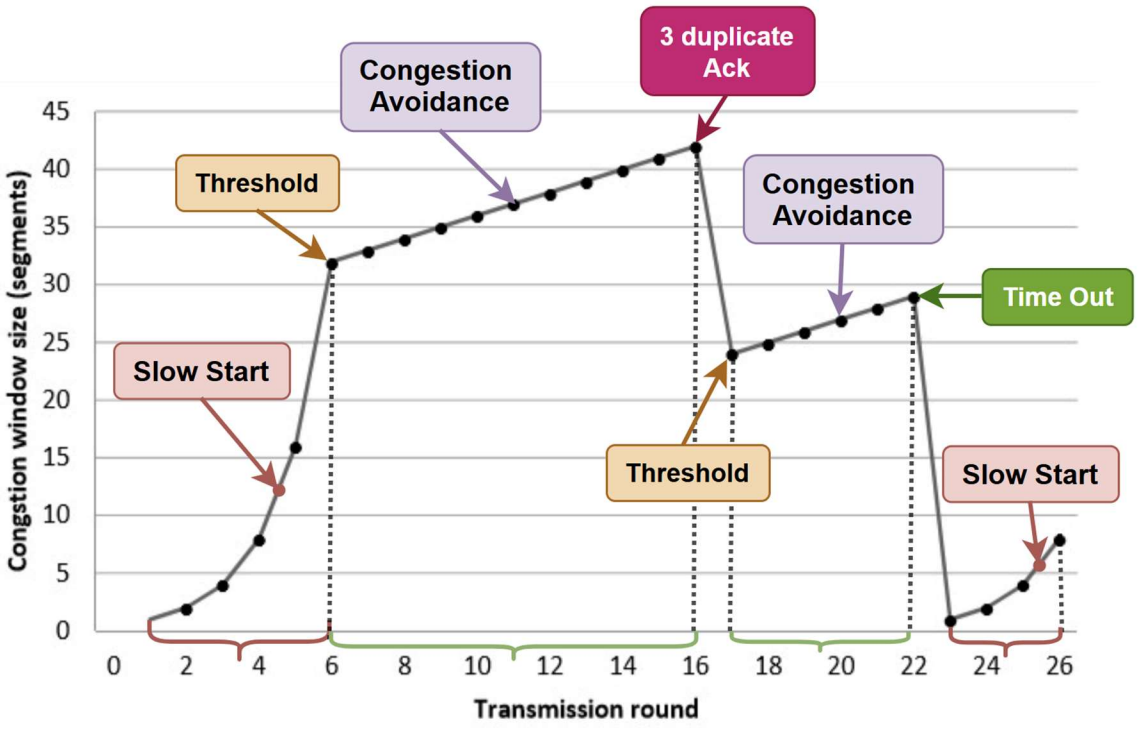
Event	Event Description	ssthresh	cwnd
1	Slow Start begins	50	$1 \rightarrow 50$
2	CA starts	50	$50 \rightarrow 56$
3	3 Dup ACK	$\frac{cwnd}{2} = \frac{56}{2} = 28$	$cwnd = ssth = 28$ (≈ 31 in graph)
4	CA resumes	31	$31 \rightarrow 34$
5	3 Dup ACK	$\frac{cwnd}{2} = \frac{34}{2} = 17$	$cwnd = ssth = 17$ (≈ 20 in graph)
6	CA resumes	20	$20 \rightarrow 26$
7	Timeout	$\frac{cwnd}{2} = \frac{26}{2} = 13$	$cwnd = 1$
8	Slow Start	12	$1 \rightarrow 13$
9	CA starts	12	$13 \rightarrow 19$

Note: Small rounding differences are normal in diagrams.

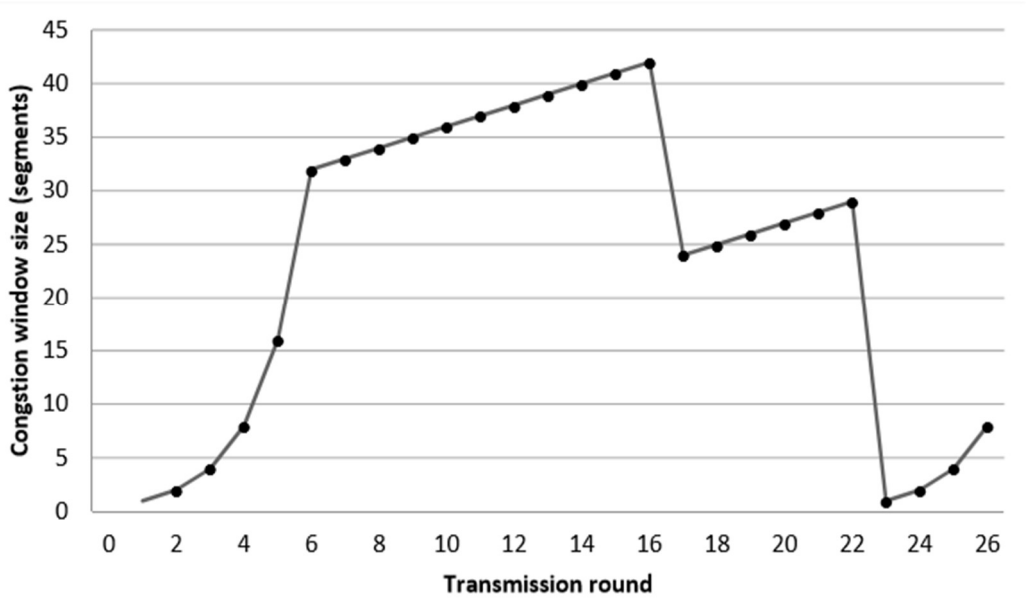
Problem3: Identify **Timeout/3 duplicate ACK** incidence. Also identify **slow start, congestion avoidance period.**



➢ **Solution:**



Problem4: Consider the following **TCP Reno scenario** (congestion window size vs. transmission round):



i. What is the **threshold of the 1st round** (with respect to the x-axis)?
ii. What is the **threshold of the 17th round** (with respect to the x-axis)?
iii. What are the **ranges of Congestion Avoidance (CA)** in the above figure (with respect to the x-axis)?
iv. What are the **coordinates (i.e., ranges of rounds) of Slow Start** in the above figure (with respect to the x-axis)?
v. What are the **reasons for the drop at the 17th and 23rd points** (with respect to the x-axis)?

➢ **Solution:**

(i)

➢ **Threshold of the 1st round (ssthresh):**

- Initially, TCP is in **Slow Start**.
- cwnd grows **exponentially** until it reaches the first threshold.
- From the graph, the exponential growth stops and linear growth starts at **round 6**, where **cwnd ≈ 32 segments**.

$$\therefore \textbf{\textit{Threshold (ssthresh) of the 1st round}} = \textbf{\textit{32 segments}}$$

(ii)

➢ Threshold of the 17th round:

- At **round 16 → 17**, cwnd drops.
- This drop is **not to 1**, so it indicates **3 duplicate ACKs** (Fast Retransmit).

$$\therefore \textbf{Threshold (ssthresh) of the } 17th \textbf{ round} = \frac{\textbf{cwnd}}{2} = \frac{42}{2} = 21$$

(iii)

➢ Ranges of Congestion Avoidance (CA):

⇨ Congestion Avoidance is where cwnd increases **linearly**.

❖ CA ranges:

- **Round 6 → Round 16**
- **Round 17 → Round 22**

(iv)

➢ Ranges of Slow Start:

⇨ Slow Start is where cwnd grows **exponentially**.

❖ SS ranges:

- **Round 1 → Round 6**
- **Round 23 → Round 26**

(v)

➢ Reasons for the drops at the 17th and 23rd rounds:

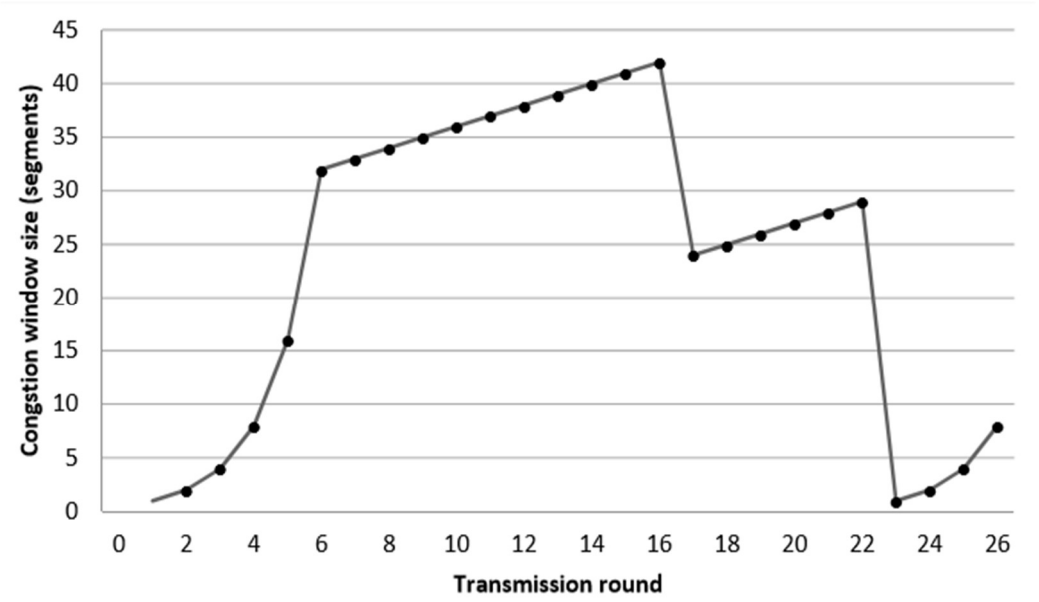
❖ Drop at 17th round:

- cwnd drops from ~42 to ~24 (not to 1)
- Indicates **3 duplicate ACKs**
- **Reason: Packet loss detected by 3 duplicate ACKs**

❖ Drop at 23rd round:

- cwnd drops from ~29 to **1**
- Indicates a **timeout.**
- **Reason: Timeout (no ACK received within RTO)**

Problem5: Consider the following **TCP Reno scenario** (congestion window size vs. transmission round):



i. Identify time intervals where TCP slow start is operating.
ii. Identify time intervals where TCP congestion-avoidance is operating
iii. After the 16th transmission round, is segment loss detected by a triple duplicate ACK or by a timeout event?
iv. After the 22nd -transmission round, is segment loss detected by a triple duplicate ACK or by a timeout event?
v. What is the ssthreshold value at the first transmission round?
vi. What is the ssthreshold value at the 18th transmission round?
vii. What is the ssthreshold value at the 24th transmission round?
viii. What will be the values of cwnd and ssthreshold if packet loss is detected after the 26th round by receipt of triple duplicate ACKs?

➢ **Solution:**

(i)

➢ Ranges of Slow Start:

⇨ Slow Start is where cwnd grows **exponentially**.

❖ **SS ranges:**

- **Round 1 → Round 6**
- **Round 23 → Round 26**

(ii)

➢ Ranges of Congestion Avoidance (CA):

⇨ Congestion Avoidance is where cwnd increases **linearly**.

❖ CA ranges:

- **Round 6 → Round 16**
- **Round 17 → Round 22**

(iii)

➢ After the 16th transmission round:

❖ Drop at 16th round:

- cwnd drops from ~42 to ~24 (not to 1)
- Indicates **3 duplicate ACKs**
- **Reason: Packet loss detected by 3 duplicate ACKs**

(iv)

➢ After the 22nd transmission round:

❖ Drop at 22nd round:

- cwnd drops from ~29 to **1**
- Indicates a **timeout.**
- **Reason: Timeout (no ACK received within RTO)**

(v)

➢ Threshold of the 1st round (ssthresh):

- Initially, TCP is in **Slow Start**.
- cwnd grows **exponentially** until it reaches the first threshold.
- From the graph, the exponential growth stops, and linear growth starts at **round 6**, where **cwnd ≈ 32 segments**.

$$\therefore \textbf{\textit{Threshold (ssthresh) of the 1st round}} = \textbf{\textit{32 segments}}$$

(vi)

➢ Threshold of the 17th round:

- Just before the drop (round 16), cwnd ≈ **42**.
- This drop is **not to 1**, so it indicates **3 duplicate ACKs** (Fast Retransmit).

$$\therefore \boldsymbol{Threshold\ (ssthresh) of\ the\ 18}th\ \boldsymbol{round} = \frac{\boldsymbol{cwnd}}{2} = \frac{42}{2} = 21$$

(vii)

➢ Threshold of the 24th round:

- Packet loss at round 23 is due to **timeout**.
- Before timeout, cwnd ≈ **29**.

$$\therefore \boldsymbol{Threshold\ (ssthresh) of\ the\ 24}th\ \boldsymbol{round} = \frac{\boldsymbol{cwnd}}{2} = \frac{29}{2} \approx 14$$

(viii)

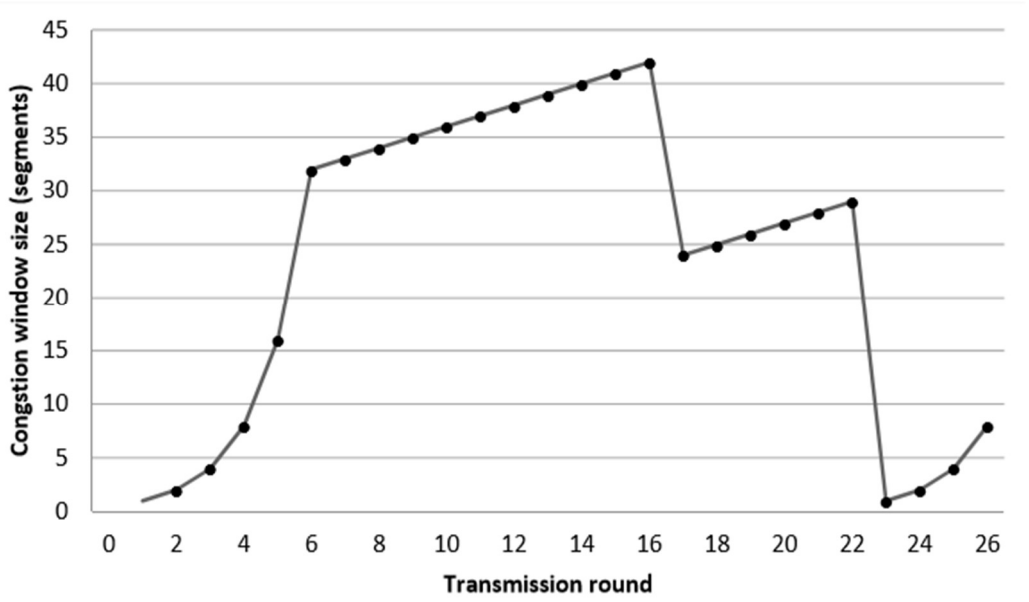
➢ Value of Threshold & cwnd at the 26th round:

- At round 26, cwnd ≈ **8**.
- Triple duplicate ACK detected.

$$\therefore \boldsymbol{Threshold\ (ssthresh) of\ the\ 26}th\ \boldsymbol{round} = \frac{\boldsymbol{cwnd}}{2} = \frac{8}{2} \approx 14$$

$$cwnd = \boldsymbol{ssthresh} = \frac{8}{4} = 4$$

Problem6: Consider the following **TCP Reno scenario** (congestion window size vs. transmission round):



i. What is the **threshold of the 1st round** (with respect to the y-axis)?
ii. What is the **threshold of the 17th round** (with respect to the y-axis)?
iii. What are the **ranges of Congestion Avoidance (CA)** in the above figure (with respect to the x-axis)?
iv. What are the **coordinates (i.e., ranges of rounds) of Slow Start** in the above figure (with respect to the x-axis)?
v. What are the **reasons for the drop at the 17th and 23rd points** (with respect to the x-axis)?

➢ **Solution:**

(i)

➢ **Threshold of the 1st round (ssthresh):**

- Initially, TCP is in **Slow Start**.
- cwnd grows **exponentially** until it reaches the first threshold.
- From the graph, the exponential growth stops and linear growth starts at **round 6**, where **cwnd ≈ 32 segments**.

$$\therefore \textbf{\textit{Threshold (ssthresh) of the 1st round}} = \textbf{32 segments}$$

(ii)

➢ Threshold of the 17th round:

- At **round 16 → 17**, cwnd drops.
- This drop is **not to 1**, so it indicates **3 duplicate ACKs** (Fast Retransmit).

$$\therefore \textbf{Threshold (ssthresh) of the } 17th \textbf{ round} = \frac{\textbf{cwnd}}{2} = \frac{42}{2} = 21$$

(iii)

➢ Ranges of Congestion Avoidance (CA):

⇨ Congestion Avoidance is where cwnd increases **linearly**.

❖ CA ranges:

- **Round 6 → Round 16**
- **Round 17 → Round 22**

(iv)

➢ Ranges of Slow Start:

⇨ Slow Start is where cwnd grows **exponentially**.

❖ SS ranges:

- **Round 1 → Round 6**
- **Round 23 → Round 26**

(v)

➢ Reasons for the drops at the 17th and 23rd rounds:

❖ Drop at 17th round:

- cwnd drops from ~42 to ~24 (not to 1)
- Indicates **3 duplicate ACKs**

Reason: Packet loss detected by 3 duplicate ACKs

❖ Drop at 23rd round:

- cwnd drops from ~29 to **1**
- Indicates a **timeout.**

Reason: Timeout (no ACK received within RTO)

TCP Buffers and Flow Control

❖ TCP Buffers:

⇨ In **TCP**, data is temporarily stored in **buffers** to manage communication between the **sender** and the **receiver**.

1. **Send buffer** → data that needs to be sent (at sender)
2. **Receive buffer** → data already received (at receiver)

❖ What is Flow Control?

⇨ **Flow control** makes sure the **sender does not send too much data** when the **receiver cannot handle it**.

- If the receiver's buffer is **full**, new packets will be **dropped**.
- To stop this, the receiver tells the sender **how much space is free**.
- This free space is called the **Receive Window (rwnd)**.

Receive Window

⇨ Created and maintained by the **receiver**
⇨ Tells the sender: "You can send only this much data"
⇨ Data outside this window is **not accepted**
⇨ Shows which **byte numbers the receiver is expecting**
⇨ If receive window = 0 → sender must **stop sending**

Send Window

⇨ Maintained by the **sender**
⇨ Contains:
  - Data already **sent but not ACKed**
  - Data **waiting to be sent**
⇨ Sender can send data **only inside the send window**

Practice Problem

Problem1: What is a connection in the light of the transport layer? Write the differences between connection-oriented and connectionless protocols in the transport layer.

⇨ In the **transport layer**, a **connection** is a **logical communication link** established between two application processes before data transmission. This connection enables **reliable data transfer**, **sequencing**, **flow control**, and **congestion control** between the sender and receiver.

➢ Difference between Connection-Oriented and Connectionless Transport Layer Protocols

Connection-Oriented Protocol	Connectionless Protocol
Connection is established before data transfer	No connection is established
Reliable data transmission	Unreliable data transmission
Uses acknowledgements (ACKs)	Does not use acknowledgements
Maintains proper order of data	Does not maintain data order
Supports flow control	Does not support flow control
Supports congestion control	Does not support congestion control
Lost packets are retransmitted	Lost packets are not retransmitted
Higher overhead	Lower overhead
Slower compared to connectionless	Faster compared to connection-oriented
Example: **TCP**	Example: **UDP**

Problem2: Between timeout and 3-duplicate acknowledgment, which one do you think indicates more severe congestion? Justify your answer.

⇨ A **timeout** indicates **more severe congestion** than **3-duplicate acknowledgments**.

- **Timeout** occurs when the sender does not receive any acknowledgment within the retransmission timeout (RTO) period. This means packets are **severely delayed or dropped**, and the network is likely **heavily congested**.
- **3-duplicate acknowledgments** occur when the receiver continues to receive packets but detects a missing one. This indicates that the network is **still delivering packets**, so congestion is **less severe**.
- In TCP behavior:
  - After a **timeout**, TCP resets the congestion window to **1** and enters **Slow Start**.
  - After **3-duplicate ACKs**, TCP only **reduces the congestion window by half** and enters **Fast Recovery**.